

École Nationale Supérieure d'Informatique pour l'Industrie et l'Entreprise

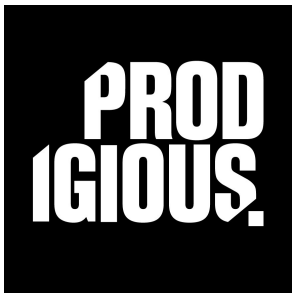
Rapport de Stage de Deuxième Année

AI and Data Engineering Internship

Projet Spielberg : Plateforme de centralisation des données opérationnelles intégrant des agents IA

Document confidentiel – Usage académique uniquement – Ne pas diffuser sans autorisation de Prodigious France.

Student : **Mouhamed Cisse**
Academic Supervisor : **Juhyun Park**



Entities : **Prodigious France, Publicis Groupe**
Address : **133 Avenue des Champs-Élysées, 75008 Paris**

Internship Supervisor : **Rony Leborgne**
Dates : **11 mai 2026 – 17 juillet 2026**

Remerciements

Je tiens à remercier Prodigious France ainsi que l'ensemble des équipes qui m'ont accueilli et accompagné tout au long de ce stage. Le projet *Spielberg* sur lequel ils m'ont donné l'opportunité de travailler m'a permis d'évoluer dans un environnement professionnel, au contact de données métier, de contraintes de sécurité et d'intégrité, de dépendances applicatives, d'exigences produit et d'échanges avec des profils variés, tout en répondant à un cahier des charges précis.

Je remercie tout particulièrement mon tuteur de stage, **Rony Leborgne**, pour son accompagnement, ses retours sur les besoins métier et la confiance qu'il m'a accordée dans la réalisation de ce projet. Sa bonne humeur constante, sa disponibilité et la pertinence de ses réponses à mes innombrables questions ont constitué une source de motivation permanente et ont grandement facilité mon intégration ainsi que ma compréhension du projet.

J'adresse également mes remerciements à **Khoudjey Traoré**, de l'équipe RH, qui a proposé ma candidature. Sans son soutien, cette expérience n'aurait pas été possible.

Je remercie également toutes les personnes ayant participé aux échanges autour des données, des besoins métier et des objectifs du projet *Spielberg*, dont l'ambition est de devenir, à terme, un outil central pour Prodigious France, puis plus largement pour le groupe Publicis.

Enfin, je tiens à remercier **Juhyun Park**, mon encadrante académique pour ce stage.

Abstract

This internship was carried out at **Prodigious France** as part of the internal **Spielberg** project, a proof of concept aimed at centralizing operational data and providing an AI-agent-based chat assistant interface for relevant collaborators. The project addresses a concrete issue : key information about people, projects, budgets, staffing, timesheets and purchase order requests is spread across several tools. This fragmentation makes data access slower, increases manual work and limits the ability to build a single operational view.

The work completed during the internship covered several complementary areas. The first part focused on the ingestion contract for the data needed across the different departments, Azure landing preparation, LLM use-case analysis, regular business and technical workshops, and internal training on compliance, data protection and anti-bribery. A second part focused on data engineering and software development : migrating JSON mock data to a local SQLite database, structuring the FastAPI backend with services, repositories and Pydantic schemas, building project and purchase order request interfaces, and integrating the Kissflow API with local persistence.

The main technical contribution concerns the AI assistant layer. The initial deterministic chatbot was progressively replaced by a first LangChain agent, then by a Router architecture, and finally by a LangGraph Supervisor architecture with specialized workers for data queries, project creation, purchase order requests and document retrieval. A RAG worker was also added to answer questions using internal project documentation. The final architecture keeps the supervisor as the single conversational entry point, delegates execution to specialized workers, and preserves structured outputs for the frontend.

The agentic layer was evaluated with LangSmith on 27 scenarios executed twice, for a total of 54 runs. The system reached a 54/54 technical success rate, a functional structure score of 48/54, corresponding to 88.9%, and no internal SQL error was observed for the data worker. These results show that the prototype provides a solid basis for a more industrialized internal data and AI platform, while still requiring further work on long-term memory, access control and cloud deployment.

Glossaire

Spielberg	POC interne combinant plateforme de données, interface web, workflows métier et assistant IA.
POC	*Proof of Concept*, prototype permettant de valider rapidement une idée ou une architecture.
CDC	Cahier des charges du projet Spielberg, utilisé pour cadrer le contexte et le périmètre fonctionnel.
Core Data Platform	Plateforme centrale destinée à unifier et exposer les données clés de l'entreprise.
Teamium	Source principale utilisée pour les bookings, ressources et données de planning.
Altair	Source ou mock utilisé pour les projets, Business Forecasts, budgets et informations financières.
BF	*Business Forecast*, référence projet ou financière utilisée dans les workflows internes.
OneClick	Source ou mock lié aux personnes, contrats, départements et absences.
rOar	Source ou mock lié aux timesheets et aux heures déclarées.
Kissflow	Plateforme de workflow utilisée pour créer et soumettre des demandes BDC.
BDC	Bon de commande ou demande fournisseur transmise via un workflow.
SQLite	Base de données locale utilisée pour structurer et persister les données du POC.
FastAPI	Framework Python utilisé pour exposer les endpoints backend.
LLM	*Large Language Model*, modèle de langage utilisé dans l'assistant IA.
Agent	Composant IA capable d'interpréter une demande et d'appeler des outils.
Tool	Fonction contrôlée appelée par un agent pour lire des données ou exécuter une action.
Orchestrator	Composant intermédiaire chargé de distribuer les demandes entre plusieurs capacités.
Supervisor	Architecture où un agent central garde la conversation et délègue à des workers spécialisés.
Worker	Sous-composant spécialisé appelé par le supervisor : data, project, bdc ou rag.
RAG	*Retrieval-Augmented Generation*, méthode combinant recherche documentaire et génération de réponse [14].
LangChain	Framework utilisé pour construire des agents, outils et chaînes LLM.
LangGraph	Framework utilisé pour modéliser le supervisor comme un graphe d'états.
LangSmith	Outil d'observabilité et d'évaluation utilisé pour analyser les exécutions de l'assistant.
MCP	*Model Context Protocol*, standard envisagé pour exposer des outils et ressources à des agents IA.
Azure	Environnement cloud Microsoft étudié pendant le stage.

Table des figures

2.1 Architecture réalisée pendant le stage : cadrage, pipeline ETL local, backend FastAPI, interfaces métier, intégration Kissflow et couche IA appliquée. 5
2.2 Écrans de l'ancien POC Spielberg avant refonte et extension de l'architecture. 7

3.1 Pipeline ETL des données Spielberg vers SQLite. 11
3.2 Architecture backend FastAPI : routes, schémas Pydantic, services, repositories, SQLite, Kissflow et agents IA 13
3.3 Adaptation du design system aux interfaces développées : dashboard, liste des projets, chatbot et détail projet. 14
3.4 Adaptation du design system pour les pages Dashboard, assistant IA, liste et détail des projets, et formulaires métier : demande BDC et création de projet. 15
3.5 Workflow applicatif de création d'un projet dans Spielberg. 16
3.6 Workflow applicatif de création et de soumission d'une demande BDC vers Kissflow. . 17
3.7 Schéma général d'un agent IA : requête utilisateur, LLM, décision, appel d'outil, résultat structuré et réponse frontend 18
3.8 Comparaison du coût en tokens entre un agent unique et des architectures multi-agents (Supervisor et swarm) et lorsque le nombre de domaines distracteurs augmente. 20
3.9 Architecture du premier agent unique : un seul agent LangChain centralise le raisonnement, le prompt global, la mémoire conversationnelle et l'accès direct aux outils. . . . 23
3.10 Première architecture multi-agent basée sur un Router : classification de l'intention puis délégation vers un agent spécialisé. 24
3.11 Graphe LangGraph de l'architecture Supervisor Spielberg. 26
3.12 Pipeline RAG documentaire intégré au Supervisor : détection de l'intention, appel du worker RAG, recherche vectorielle, sélection des sources et réponse structurée côté frontend. 30
3.13 Taux d'erreur des traces LangSmith pendant les évaluations du supervisor. 32
3.14 Latence des appels LLM observée dans LangSmith : percentiles P50 et P99. 32
3.15 Latence médiane par nœud LangGraph du supervisor. 33
3.16 Nombre d'appels par nœud LangGraph dans les traces LangSmith. 34
3.17 Tokens d'entrée par trace LangSmith. 34
3.18 Coût médian par trace dans LangSmith. 35

B.1 Organigramme Prodigious/Publicis. 41

Table des matières

Remerciements i

Abstract ii

Glossaire iii

Liste des figures iv

1 Introduction : environnement de travail et stage 1

1.1 Présentation de l'entreprise : Prodigious France . . . 1
1.2 Département de travail . . . 1
1.3 Contexte et problématique . . . 1
1.4 Objectifs du stage . . . 2
1.5 Contributions principales . . . 2

2 Contexte et notions préliminaires 4

2.1 Spécifications du stage . . . 4
2.2 Méthodes et outils utilisés dans l'entreprise . . . 5
2.3 Architecture logicielle à étendre . . . 6

3 Solution implémentée et contributions 8

3.1 Étude des solutions possibles . . . 8
3.2 Solution implémentée . . . 9
3.2.1 Détails techniques . . . 9
Axe 1 – Cadrage, préparation data et développement applicatif . . . 9
Axe 2 – Implémentation des agents IA, architecture agentique et évaluation . . . 17
3.2.2 Résultats des expérimentations ou tests techniques . . . 31
3.2.3 Discussion des contributions . . . 35

4 Conclusion et perspectives 37

4.1 Conclusion . . . 37
4.2 Enseignements tirés . . . 37
4.3 Prochaines étapes . . . 38

A DD&RS - Développement durable et responsabilité sociale **39**
A.1 Impacts environnementaux . . . 39
A.1.1 Liés à l'activité de l'entreprise . . . 39
A.1.2 Liés au fonctionnement de l'entreprise . . . 39
A.1.3 Liés à l'activité de l'équipe ou du département . . . 39
A.2 Actions pour minimiser les impacts . . . 40
A.2.1 Actions mises en place par l'entreprise . . . 40
A.2.2 Actions intégrées dans les stratégies de l'entreprise . . . 40
A.2.3 Canaux de communication dans l'environnement de travail . . . 40
A.2.4 Actions liées au stage et aux stratégies de l'entreprise . . . 40

B Structure de l'entreprise **41**

C Détails techniques **42**
C.1 Cache et réduction des chargements inutiles . . . 42
C.2 Cache applicatif sur les projets . . . 42
C.3 Singleton du supervisor . . . 43
C.4 Garde-fous avant appel LLM . . . 43
C.5 Accès SQL read-only . . . 44
C.6 Finalisation déterministe après worker . . . 44
C.7 RAG et vector store persistant . . . 44
C.8 Worker RAG et sources frontend . . . 45
C.9 Intégration BDC et traçabilité . . . 45

Bibliographie **46**

Chapitre 1

Introduction : environnement de travail et stage

1.1 Présentation de l'entreprise : Prodigious France

Prodigious France est l'entité de production et d'exécution créative de Publicis Groupe. Elle accompagne les différentes agences du groupe dans la réalisation de campagnes publicitaires en mettant à disposition des expertises variées telles que la création, la production, la post-production, le digital, la gestion de projet et les opérations. Cette activité repose sur une collaboration étroite entre de nombreux métiers et sur l'exploitation quotidienne d'un volume important de données opérationnelles.

C'est dans ce contexte qu'a été développé le projet *Spielberg*, une plateforme interne ayant pour ambition de devenir la *Single Source of Truth* de Prodigious. Son objectif est de centraliser au sein d'un environnement unique les données relatives aux collaborateurs, aux projets et aux informations financières, aujourd'hui réparties entre plusieurs applications.

Au-delà de la simple centralisation des données, Spielberg vise à remplacer un écosystème d'outils fragmenté par une plateforme unifiée permettant d'améliorer la qualité et la cohérence des informations, de réduire les ressaisies manuelles et d'accélérer les processus métier. La plateforme se démarque des data lakes classiques avec l'intégration d'agents IA capables de répondre en langage naturel aux questions opérationnelles et décisionnelles des collaborateurs, facilitant ainsi l'accès aux informations et l'aide à la prise de décision.

1.2 Département de travail

Ce stage s'est déroulé au sein du département **Développement**, rattaché au pôle **Digital** de Prodigious France, où j'occupais le poste de *Assistant Lead Automation*. Dans le cadre du projet Spielberg, j'ai toutefois été amené à collaborer avec de nombreux autres pôles métiers, notamment le staffing, la finance, les ressources humaines, l'IT, la sécurité ainsi que les équipes projet.

Cette collaboration transversale s'est traduite par une participation régulière à des réunions et ateliers avec ces différents acteurs afin d'identifier les besoins métier, de comprendre les sources de données disponibles, de prendre en compte les contraintes techniques, éthiques, opérationnelles et sécuritaires, ainsi que de définir les fonctionnalités attendues du POC.

1.3 Contexte et problématique

Le cahier des charges de Spielberg décrit une situation initiale où les données opérationnelles clés sont fragmentées entre plusieurs outils internes et externes au groupe, fichiers Excel etc. Cette fragmentation ralentit les décisions de staffing, augmente la charge manuelle et rend certaines réconciliations difficiles.

Les problèmes principaux concernaient la recherche de disponibilité, encore trop manuelle, la visibilité dispersée sur les projets, les budgets et les BF, ainsi que la lenteur de certains processus comme la création de projet, le passage au planning ou la préparation des timesheets. Les tableaux de bord reposaient encore largement sur des exports et il manquait un point d'accès unique permettant de consulter les données opérationnelles de manière cohérente.

1.4 Objectifs du stage

L'objectif principal de ce stage était de contribuer au développement du projet *Spielberg*, une plateforme interne de centralisation et d'exploitation des données de Prodigious France. Pour atteindre cet objectif, il m'a été demandé de comprendre les besoins fonctionnels à partir du cahier des charges, du code du POC existant et des échanges avec les équipes métier, de participer à la conception et au développement de nouvelles fonctionnalités, ainsi que de contribuer à l'intégration et à la valorisation des données issues de plusieurs applications internes.

Le stage visait également à développer des services backend avec FastAPI, à concevoir des interfaces permettant notamment la création de projets et la gestion des demandes de BDC, ainsi qu'à mettre en œuvre une architecture d'agents conversationnels fondée sur les modèles de langage (LLM), modélisée avec LangChain et LangGraph. Cette architecture devait rester encadrée par des guardrails déterministes afin de protéger l'intégrité des données. Enfin, une partie importante du travail consistait à étudier les approches récentes en IA agentique, afin de proposer une solution modulaire, documentée et adaptée.

1.5 Contributions principales

Mes contributions se sont articulées autour de trois axes : le cadrage métier, le développement applicatif et l'évolution de l'assistant Spielberg vers une architecture agentique plus robuste.

Les principales réalisations peuvent être résumées ainsi :

— cadrage métier avec le contrat d'ingestion, la préparation Azure, la préparation de la couche LLM et l'analyse du CDC ;
— participation à des réunions et ateliers réguliers avec les pôles métier, dont certains échanges avec des interlocuteurs internationaux ;
— montée en compétence sur Azure, la conformité, la protection des données, l'anti-bribery, les LLM, le NLP, le RAG, Hugging Face, LangChain et LangGraph ;
— migration des données de fichiers JSON vers SQLite, structuration du backend FastAPI et création d'endpoints métier ;
— adaptation du frontend au design system Prodigious, avec les interfaces de création projet et de demande BDC intégrée à Kissflow ;
— évolution de l'assistant depuis un chat déterministe vers un agent LangChain, puis vers une architecture Router/Supervisor avec workers spécialisés et RAG documentaire.

La couche IA a été évaluée avec LangSmith sur 27 scénarios répétés deux fois, soit 54 exécutions. Les résultats obtenus sont de 54/54 exécutions en succès technique, un score fonctionnel structurel de 48/54, soit 88,9 %, et un score textuel strict de 45/54, soit 83,3 %. Le worker `data` atteint 20/20 réussites fonctionnelles, avec 22/22 sorties SQL contenant une table exploitable et aucune erreur SQL interne détectée. Les workers `project` et `bdc` obtiennent respectivement 10/14 et 10/12 en score fonctionnel structurel ; les écarts restants concernent surtout les reprises de contexte multi-tours. La

latence médiane observée est de 3,01 s, la latence moyenne de 4,53 s et le p95 de 10,56 s. À titre indicatif, pour une passe de 27 demandes, le temps agent observé est d'environ 2,04 minutes. La comparaison avec une durée manuelle comprise entre 2 h 40 et 5 h 26 repose sur mes estimations personnelles et ne constitue pas une mesure officielle validée par l'entreprise.

Chapitre 2

Contexte et notions préliminaires

2.1 Spécifications du stage

Le stage s'est inscrit dans le périmètre du projet *Spielberg*, dont l'objectif est de faire évoluer un POC interne vers une plateforme plus structurée pour centraliser et exploiter les données opérationnelles de Prodigious. Le périmètre retenu devait permettre de valider les briques principales avant une éventuelle généralisation : socle de données local, backend FastAPI, interfaces métier, intégration Kissflow pour la demande de BDC, création de projet et assistant IA capable d'interagir avec les données, les outils disponibles et la documentation du projet.

Le travail demandé s'inscrivait donc dans le cadre de l'ingénierie IA et du développement logiciel. Il ne consistait pas seulement à ajouter des fonctionnalités isolées, mais à contribuer à une trajectoire d'industrialisation progressive. Les développements réalisés devaient rester compatibles avec les contraintes métier, la qualité des données, la sécurité, la traçabilité des actions sensibles et l'usage futur d'une infrastructure cloud.

Pour la version académique du rapport, les exemples, captures et extraits doivent rester compatibles avec les règles de confidentialité de l'entreprise. Les données présentées sont donc à considérer comme fictives, mockées ou anonymisées, et les captures ne doivent pas faire apparaître de nom de salarié, client, projet réel, URL interne sensible, token, secret ou identifiant technique confidentiel. Les extraits de code conservés dans les annexes sont limités aux parties non sensibles et communicables dans un cadre académique.

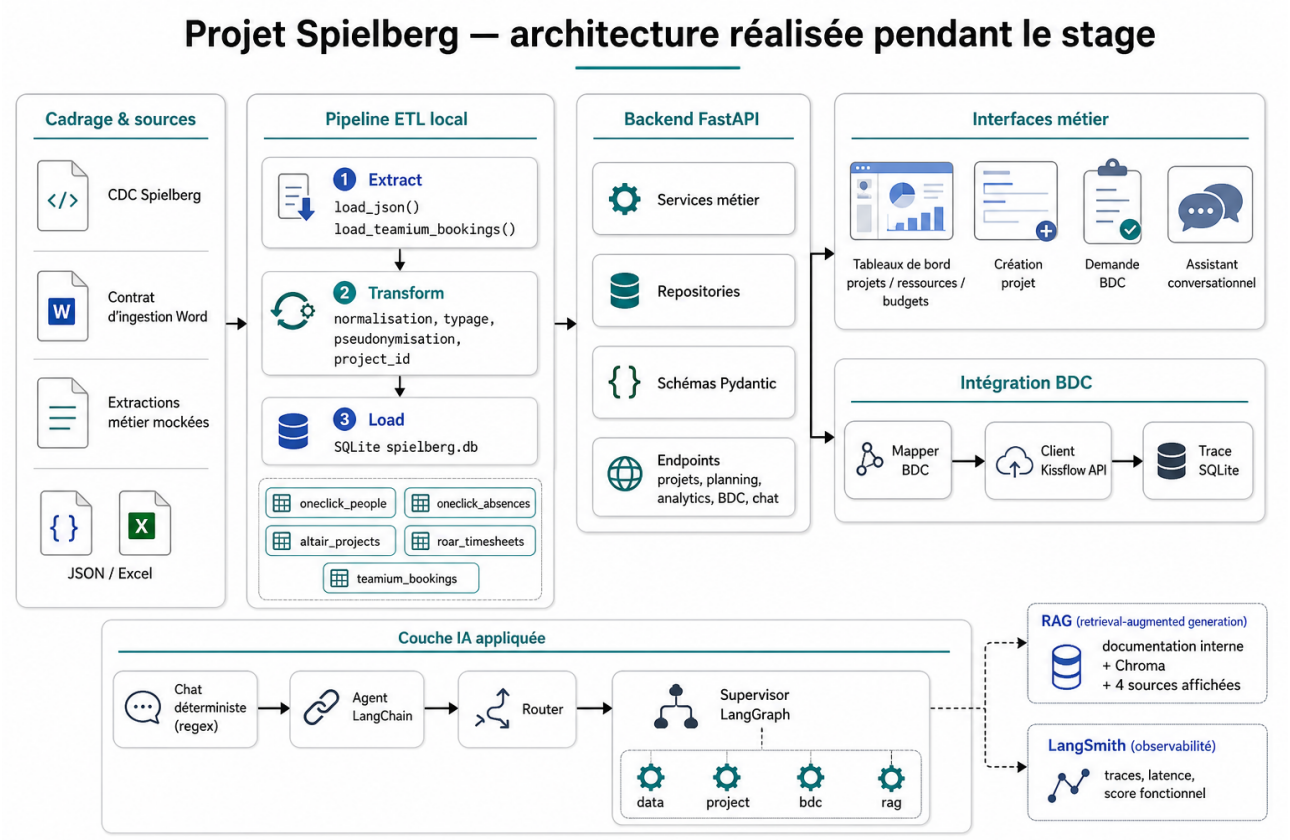


FIGURE 2.1 – Architecture réalisée pendant le stage : cadrage, pipeline ETL local, backend FastAPI, interfaces métier, intégration Kissflow et couche IA appliquée.

2.2 Méthodes et outils utilisés dans l'entreprise

La méthodologie de travail suivie pendant le stage s'est appuyée sur plusieurs éléments de cadrage. Le premier était le cahier des charges du projet *Spielberg*, qui définissait la vision cible d'une plateforme jouant le rôle de *Single Source of Truth* pour les données opérationnelles de Prodigious. Ce CDC a ensuite été affiné au fil des échanges avec les équipes métier et transformé progressivement en tâches exécutables dans JIRA, chacune associée à un besoin, un périmètre et des critères de validation.

Le travail s'est également appuyé sur des réunions et ateliers réguliers avec différents pôles, parfois avec des interlocuteurs internationaux. Ces échanges permettaient de clarifier les besoins, de réajuster le périmètre fonctionnel et de valider les choix au fur et à mesure. En complément, j'ai proposé certaines idées issues de lectures techniques ou de documentations, notamment autour des agents IA, du RAG, de LangChain et de LangGraph ; ces propositions étaient ensuite discutées puis implémentées après validation par mon tuteur.

Le développement a été mené de manière incrémentale, avec des cycles courts comprenant l'implémentation, les tests manuels, la correction des écarts observés, la documentation technique et l'analyse des résultats obtenus.

Les outils mobilisés couvraient à la fois la coordination, le développement, la préparation cloud et l'évaluation :

— Microsoft Teams, JIRA et Git Azure DevOps pour les échanges, le suivi des tâches et le version-

nement ;
— Codex comme appui au développement, à l'exploration du code et à la relecture avant validation ;
— FastAPI, Pydantic, SQLite, HTML, CSS, JavaScript et le design system consulté dans Figma pour la partie applicative ;
— Docker et Microsoft Azure pour préparer une trajectoire de déploiement plus reproductible et industrialisable ;
— Hugging Face, LangChain, LangGraph et LangSmith pour la partie IA, les agents, le supervisor, l'observabilité et l'évaluation ;
— Kissflow API et les formations internes pour intégrer le workflow BDC tout en respectant le cadre conformité, protection des données et anti-bribery.

2.3 Architecture logicielle à étendre

Au début du stage, Spielberg était un POC fonctionnel, construit autour d'un backend `FastAPI`, d'un frontend web léger, de données simulées principalement issues de fichiers JSON et Excel, ainsi que d'un chatbot déterministe reposant sur des règles et des expressions régulières. L'utilisation de l'intelligence artificielle y était encore limitée : elle servait essentiellement à reformuler les requêtes des utilisateurs afin de faciliter l'extraction des entités clés, telles que les dates, les rôles ou les compétences, notamment dans le cadre des requêtes de staffing.

Cette architecture était adaptée à une première démonstration, car elle permettait de valider rapidement les principaux parcours utilisateurs ainsi que l'intérêt métier de l'assistant. En revanche, pour évoluer vers une application plus robuste et maintenable, plusieurs limites devaient être levées : une structuration insuffisante des données, une logique métier encore dispersée, l'absence d'une couche d'accès aux données clairement isolée, une interface non encore alignée sur un design system précis, ainsi que la difficulté à gérer des conversations multi-tours avec des réponses structurées pour le frontend. Le fil de la discussion avec l'assistant IA n'était pas encore complètement affiché.

Elle a aussi servi de socle expérimental pour valider rapidement la valeur métier de Spielberg. Elle permettait déjà de naviguer dans certaines données projet, de tester une première interaction conversationnelle et de montrer le potentiel d'un assistant interne. Les écrans suivants illustrent cet état initial : une interface encore légère, centrée sur la démonstration fonctionnelle, avant la structuration progressive du backend, du frontend, des données et de l'architecture agentique.

Le travail réalisé a donc consisté à faire évoluer ce POC vers une architecture plus modulaire, organisée par couches et par responsabilités. Les principales extensions apportées sont les suivantes :

— migration des données simulées au format JSON vers une base SQLite exploitable par le backend et les agents de consultation ;
— ajout de services backend, repositories et schémas Pydantic pour séparer l'accès aux données, la logique métier et les contrats d'API ;
— développement d'endpoints métier autour des projets, des ressources, des budgets, des demandes BDC et de l'assistant conversationnel ;
— adaptation des interfaces frontend au design system Prodigious, avec un tableau de bord, des vues projet et des formulaires métier ;
— intégration Kissflow pour connecter le processus de demande BDC à un workflow externe ;
— évolution de la couche IA depuis un chatbot déterministe vers un agent LangChain, puis vers une architecture Router et enfin Supervisor avec workers spécialisés et évaluation LangSmith.

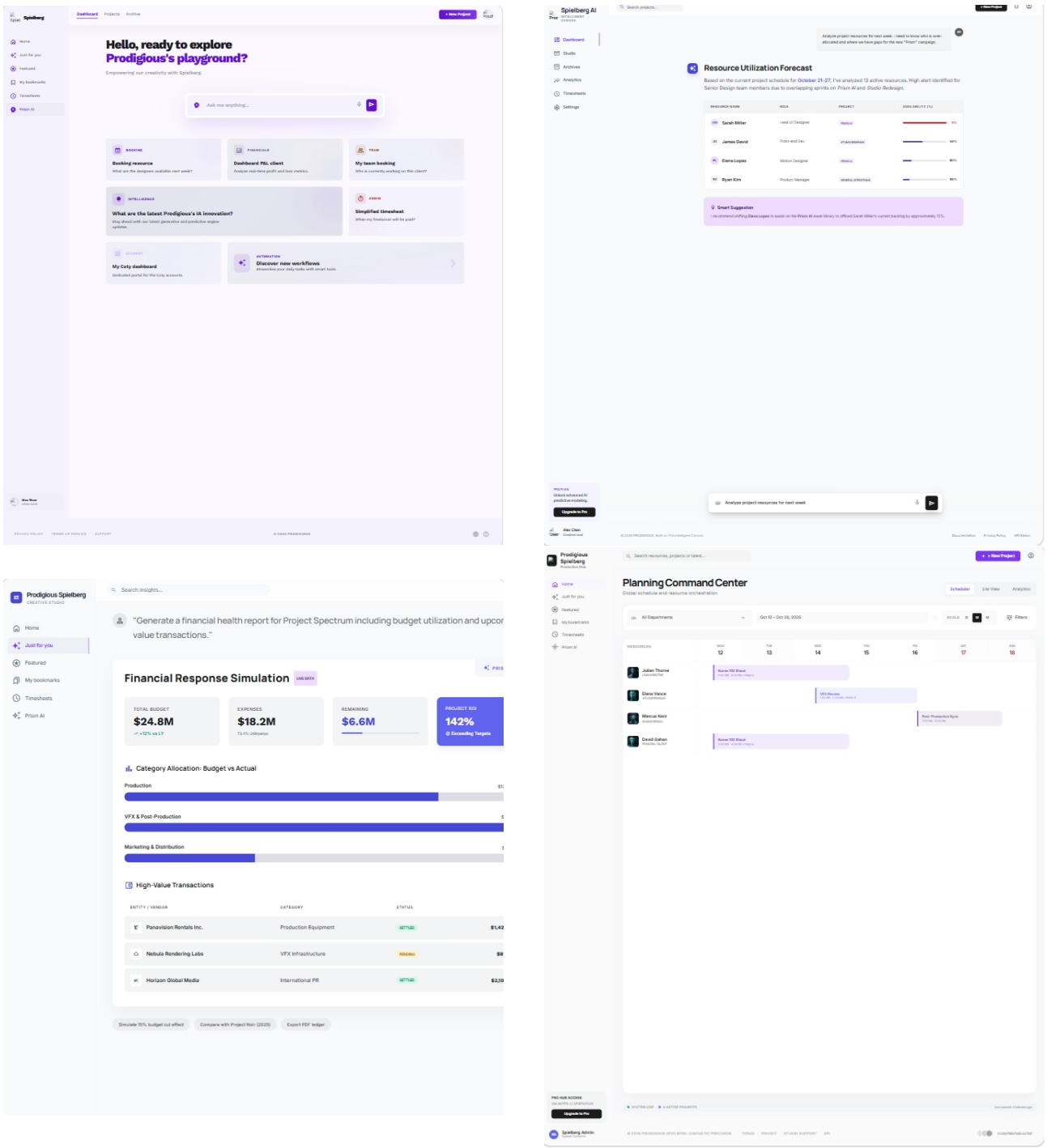


FIGURE 2.2 – Écrans de l'ancien POC Spielberg avant refonte et extension de l'architecture.

Chapitre 3

Solution implémentée et contributions

3.1 Étude des solutions possibles

Plusieurs choix techniques ont été étudiés pendant le stage afin de faire évoluer Spielberg d'un POC basé sur des mocks et un chat déterministe vers une application plus structurée, capable d'exposer des données métier, des interfaces opérationnelles et un assistant IA spécialisé.

Table 3.1 – Synthèse des choix techniques étudiés pendant le stage.

Sujet	**Options étudiées**	**Choix retenu**
Stockage	Fichiers JSON, SQLite, PostgreSQL	SQLite pour structurer le POC et préparer une trajectoire vers PostgreSQL.
Backend	Logique directe, services isolés, repositories	FastAPI avec services métier, schémas Pydantic et repositories.
Chat	Regex déterministe, agent unique, Router, Supervisor	Passage progressif vers une architecture Supervisor avec workers spécialisés.
Agents IA	Agent LangChain simple, multi-agents, LangGraph	LangChain pour les premiers agents, puis LangGraph pour formaliser le Supervisor.
RAG	Pas de RAG, recherche simple, base vectorielle locale	RAG local avec Chroma pour interroger la documentation interne du projet.
Frontend	Interface POC légère, composants réutilisables, design system	Application du design system Prodigious sur les écrans métier.
Intégration BDC	Simulation locale, API directe, service dédié	Module BDC avec schémas, mapper, client Kissflow et persistance locale.
Déploiement cible	Exécution locale, conteneurisation, Azure	Préparation d'une trajectoire Docker et Azure pour une industrialisation future.

Ces choix ont été guidés par les contraintes du POC : démontrer rapidement la valeur métier, rester compréhensible pour les équipes, éviter une architecture trop lourde dès le départ, tout en préparant une trajectoire plus industrialisable et scalable.

3.2 Solution implémentée

3.2.1 Détails techniques

Cette section présente les différentes phases réalisées pendant le stage présentée de façon chronologique. Les premières phases correspondent au cadrage, à la préparation documentaire, aux choix applicatifs et au développement des briques métier. Les phases suivantes constituent le cœur technique du stage, centré sur l'implémentation des agents IA, les choix d'architecture agentique, le RAG documentaire et l'évaluation.

Axe 1 – Cadrage, préparation data et développement applicatif

Phase 1 & 2 – Cadrage préliminaire : ingestion, Azure et couche LLM

Avant les développements applicatifs, une première phase de cadrage a été menée afin de clarifier le périmètre du projet, les données nécessaires, les contraintes d'exploitation et les usages réalistes de l'assistant IA. Cette phase n'était pas une phase d'implémentation directe, mais une étape préparatoire indispensable pour éviter de développer sur des hypothèses implicites.

Elle s'est articulée autour de trois livrables principaux. Le premier est le contrat d'ingestion Word. Le deuxième est un document de préparation à l'atterrissage Azure, qui a permis d'anticiper les composants nécessaires à une future industrialisation du POC. Le troisième est un document de préparation de la couche LLM / langage naturel, visant à cadrer les usages pertinents des modèles de langage dans Spielberg. Ces éléments ont permis de mieux relier les besoins métier du CDC aux choix d'architecture réalisés ensuite. Les livrables produits ont surtout servi à transformer un besoin encore large en éléments exploitables pour le développement : sources à intégrer, contraintes d'accès, choix d'hébergement, usages réalistes des LLM et garde-fous à prévoir.

Cette phase a également été complétée par des formations et recherches personnelles : environnement Microsoft Azure, formations internes de conformité, protection des données et anti-bribery, ainsi que montée en compétence sur les LLM, le NLP, le RAG, Hugging Face, LangChain et LangGraph.

Contrat d'ingestion :

Le contrat d'ingestion a constitué le premier support de cadrage data. Pour chaque source, il précisait l'identification du système, le responsable côté métier ou source, le mode d'accès attendu, le format des fichiers ou des flux, la fréquence de mise à disposition et le type d'extraction. Il décrivait aussi les identifiants métier utiles aux jointures, les champs attendus, la source maître, les règles de réconciliation, le niveau de sensibilité, les dépendances, la criticité et le statut de validation. Cette structuration a permis de remplacer des hypothèses implicites par un vocabulaire commun entre besoin métier, modèle local, mocks et futurs endpoints backend.

Préparation de l'atterrissage Azure :

Le document de préparation Azure a servi à anticiper la trajectoire d'industrialisation du POC. Les principaux volets étudiés concernaient l'hébergement du backend FastAPI avec Container Apps ou App Service, l'hébergement du frontend statique avec Static Web Apps, une cible de persistance plus robuste avec PostgreSQL Flexible Server après la phase SQLite, le stockage de documents ou d'exports, la centralisation des secrets avec Key Vault et l'observabilité avec Application Insights. Ces composants représentent les options étudiées lors de la préparation Azure et non nécessairement l'architecture finale déployée. L'architecture définitive reste soumise aux décisions GSO et RE :SOURCES et peut évoluer lors du provisioning ; la dernière trajectoire validée distingue notamment les couches Front, App et Data, s'appuie sur des réseaux privés, Key Vault et du monitoring, tandis que Blob Storage n'est pas

retenu dans le MVP actuel.

Préparation de la couche LLM :

Le document de préparation LLM a cadré les usages réalistes du langage naturel dans Spielberg. Les cas envisagés concernaient notamment la recherche de disponibilités, l'assistance au staffing, l'aide à la compréhension des timesheets, la lecture guidée des KPI, le routage des requêtes et la production de réponses structurées. Le principe retenu est que le LLM comprend, reformule, classe et synthétise, tandis que les calculs métier et les actions sensibles restent portés par le backend. Les garde-fous associés sont donc essentiels : ne pas inventer de disponibilités, rattacher les recommandations aux données source, demander une clarification en cas d'ambiguïté, limiter l'exposition de données sensibles et garder une validation humaine avant toute action engageante.

Application : Ces travaux préliminaires ont permis de consolider les choix d'architecture du projet avant l'implémentation. Ils ont notamment confirmé l'intérêt d'une architecture agentique, tout en tenant compte des limites propres aux LLM : risque d'hallucination, ambiguïtés d'intention, besoin de garde-fous, traçabilité des sources et séparation entre raisonnement en langage naturel et calculs métier déterministes. Ils ont également mis en évidence la nécessité d'un RAG documentaire pour exploiter la documentation interne du projet, ainsi qu'une première réflexion sur l'industrialisation future du POC dans un environnement Azure.

Phase 3 – Pipeline ETL et migration des données vers SQLite

Après le cadrage des sources, la première étape applicative a consisté à mettre en place un pipeline ETL pour migrer des données mockées, construites à partir d'extractions proches des vraies données métier, vers une base SQLite locale. L'objectif était de passer d'une logique de fichiers isolés à un socle relationnel exploitable par le backend FastAPI, les tableaux du frontend et le worker `data` de l'assistant IA.

Le pipeline suit une logique *Extract, Transform, Load*. La phase d'extraction récupère les données depuis des fichiers JSON et Excel représentant les principaux domaines fonctionnels du POC. La phase de transformation normalise les valeurs manquantes, convertit les dates, convertit les champs numériques, stabilise certains identifiants et applique une pseudonymisation des collaborateurs lorsque les données proviennent des bookings. La phase de chargement insère ensuite les données dans SQLite selon un modèle relationnel plus adapté aux jointures et aux requêtes métier.

Le script construit d'abord une base temporaire `spielberg_next.db`, active les clés étrangères, crée les tables métier, charge les données transformées, affiche les volumes chargés, puis publie la base finale sous le nom `spielberg.db`. Cette stratégie de construction intermédiaire évite de remplacer la base active si une erreur survient pendant le chargement.

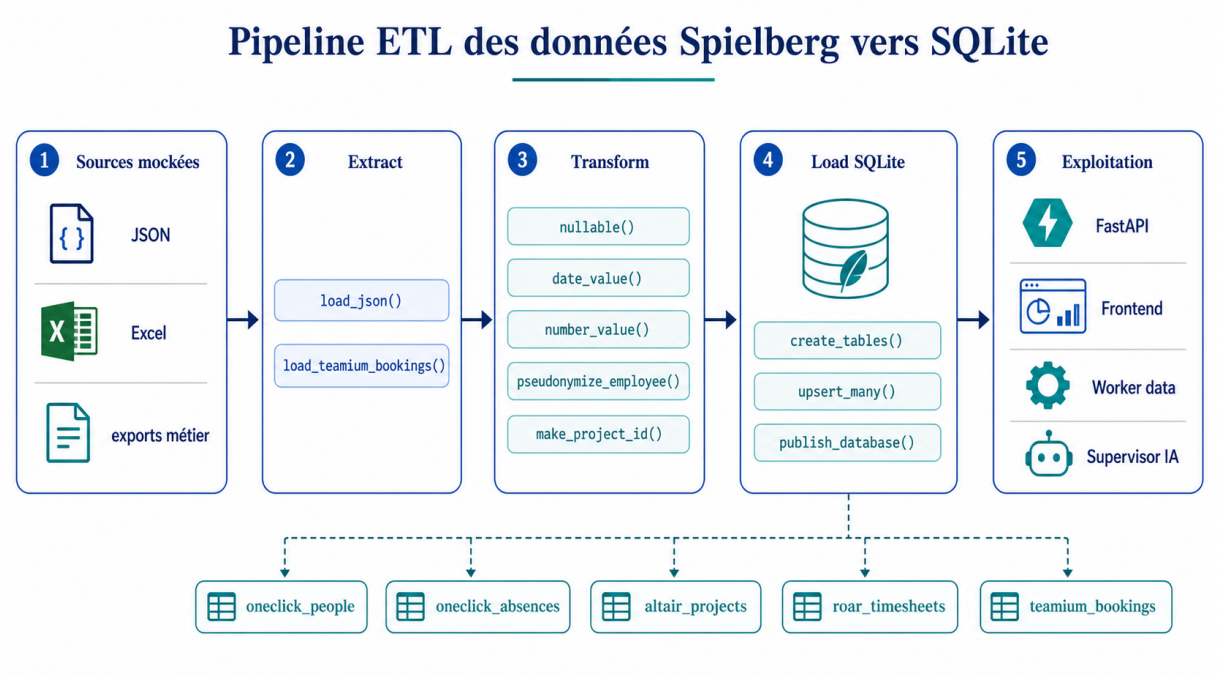


FIGURE 3.1 – Pipeline ETL des données Spielberg vers SQLite.

Cette phase a transformé les mocks initiaux en un socle de données plus structuré. Elle a rendu possibles des requêtes SQL plus fiables, la construction d'endpoints backend plus propres, l'affichage de tableaux côté frontend et l'interrogation des données par l'agent `data`. Elle prépare également une trajectoire future vers PostgreSQL : SQLite reste adapté au POC local, mais le passage par un modèle relationnel, des clés et des services backend rend la transition vers une base plus industrielle plus réaliste.

Phase 4 – Backend FastAPI, services métier et schémas Pydantic

Principe technique :

Une API backend sert de couche d'intermédiation entre les données, le frontend et les agents IA. Elle évite que l'interface utilisateur ou l'assistant conversationnel accèdent directement à la base de données. Cette séparation permet de centraliser la logique métier, de valider les entrées, de contrôler les formats de sortie, de gérer les erreurs et de préparer plus facilement une évolution vers une architecture plus industrielle.

Dans ce type d'architecture, `FastAPI` est particulièrement adapté car il permet de créer rapidement des endpoints HTTP en Python, tout en s'appuyant sur le typage et sur les modèles `Pydantic`. Les schémas Pydantic jouent le rôle de contrat entre le backend, le frontend et les agents : ils décrivent les champs attendus, les types autorisés, les valeurs optionnelles, les réponses retournées et les erreurs possibles. Ils facilitent également la génération automatique de documentation OpenAPI, utile pour tester et partager les endpoints.

Le backend peut ainsi être organisé en plusieurs couches complémentaires : les routes FastAPI exposent les endpoints HTTP, les schémas Pydantic valident les requêtes et structurent les réponses, les services métier portent la logique fonctionnelle, les repositories isolent l'accès aux données et les intégrations externes encapsulent les appels vers des API tierces.

Application dans Spielberg :

Dans Spielberg, cette organisation a été appliquée pour structurer progressivement le backend autour des principaux besoins du POC. Le backend `FastAPI` expose des endpoints pour consulter les projets, les ressources, le planning, les analytics, les timesheets, les demandes BDC et l'assistant conversationnel. Les données issues de SQLite sont donc exploitées à travers des services dédiés plutôt que manipulées directement depuis le frontend.

Les schémas `Pydantic` ont été utilisés pour formaliser les contrats d'entrée et de sortie, notamment pour les objets liés aux projets et aux demandes BDC. Cette étape est importante car les formulaires frontend et les agents IA produisent tous deux des données structurées qui doivent être validées avant d'être utilisées. Par exemple, une demande BDC ou une création de projet ne peut pas reposer uniquement sur du texte libre généré par un LLM : les champs doivent être contrôlés, typés et transmis au backend dans un format stable.

Le backend joue également un rôle central dans l'intégration avec Kissflow. La logique est séparée entre le schéma de demande, le service métier, le mapper et le client API Kissflow. Cette séparation permet de transformer les champs internes de Spielberg vers le format attendu par Kissflow, tout en conservant une trace locale de la demande dans SQLite.

Enfin, le backend sert aussi de point d'appui aux agents IA. Le worker `data` s'appuie sur les données structurées pour répondre aux questions métier, tandis que les workers `project` et `bdc` produisent des payloads compatibles avec les formulaires frontend. Cette architecture connecte dans un même système les données, les interfaces métier et l'assistant conversationnel, tout en gardant une frontière claire entre compréhension LLM et exécution déterministe.

```python
from pydantic import BaseModel
from typing import Optional

class BdcRequest(BaseModel):
    supplier_name: str
    amount: float
    currency: str = "EUR"
    requester_email: str
    project_id: Optional[str] = None
    description: Optional[str] = None
```

Listing 3.1 – Exemple simplifié de contrat Pydantic pour une demande métier.

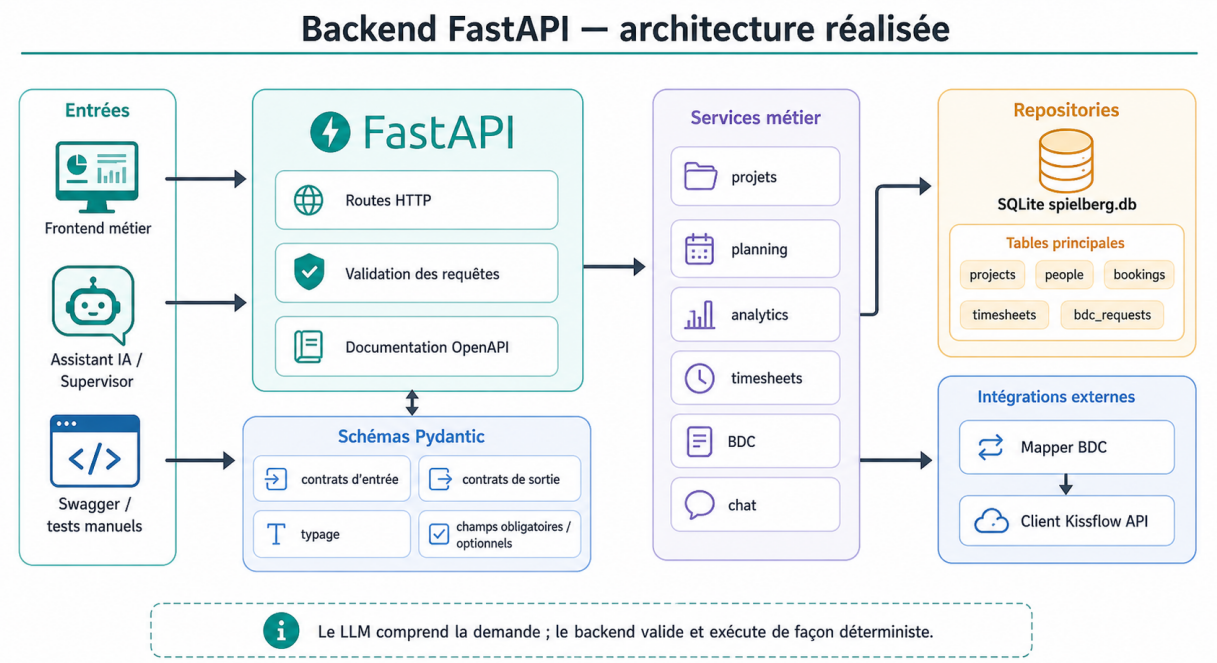


FIGURE 3.2 – Architecture backend FastAPI : routes, schémas Pydantic, services, repositories, SQLite, Kissflow et agents IA

Phase 5 – Frontend, design system et interfaces métier

Principe technique :

Un design system permet d'unifier l'apparence et le comportement d'une application : couleurs, typographies, espacements, composants, boutons, formulaires, tableaux, états visuels et règles d'ergonomie. Dans un POC métier, il joue un rôle important car il rend l'outil plus lisible, plus crédible et plus facilement appropriable par les utilisateurs.

Application dans Spielberg :

Le design system utilisé dans Spielberg avait été défini par d'autres membres de l'équipe. Mon rôle n'a donc pas été de créer l'identité graphique ou les règles visuelles principales, mais de les adapter et de les appliquer aux écrans développés pendant le stage. Ce travail a consisté à reprendre les composants, les couleurs, les espacements et les conventions visuelles existantes afin d'assurer une cohérence entre les nouvelles interfaces et le reste du POC.

Cette adaptation a concerné plusieurs écrans métier : tableau de bord, liste des projets, détail d'un projet, assistant conversationnel, formulaire de création de projet et formulaire de demande BDC. L'objectif était de conserver une expérience homogène tout en intégrant de nouvelles fonctionnalités connectées au backend FastAPI et aux agents IA.

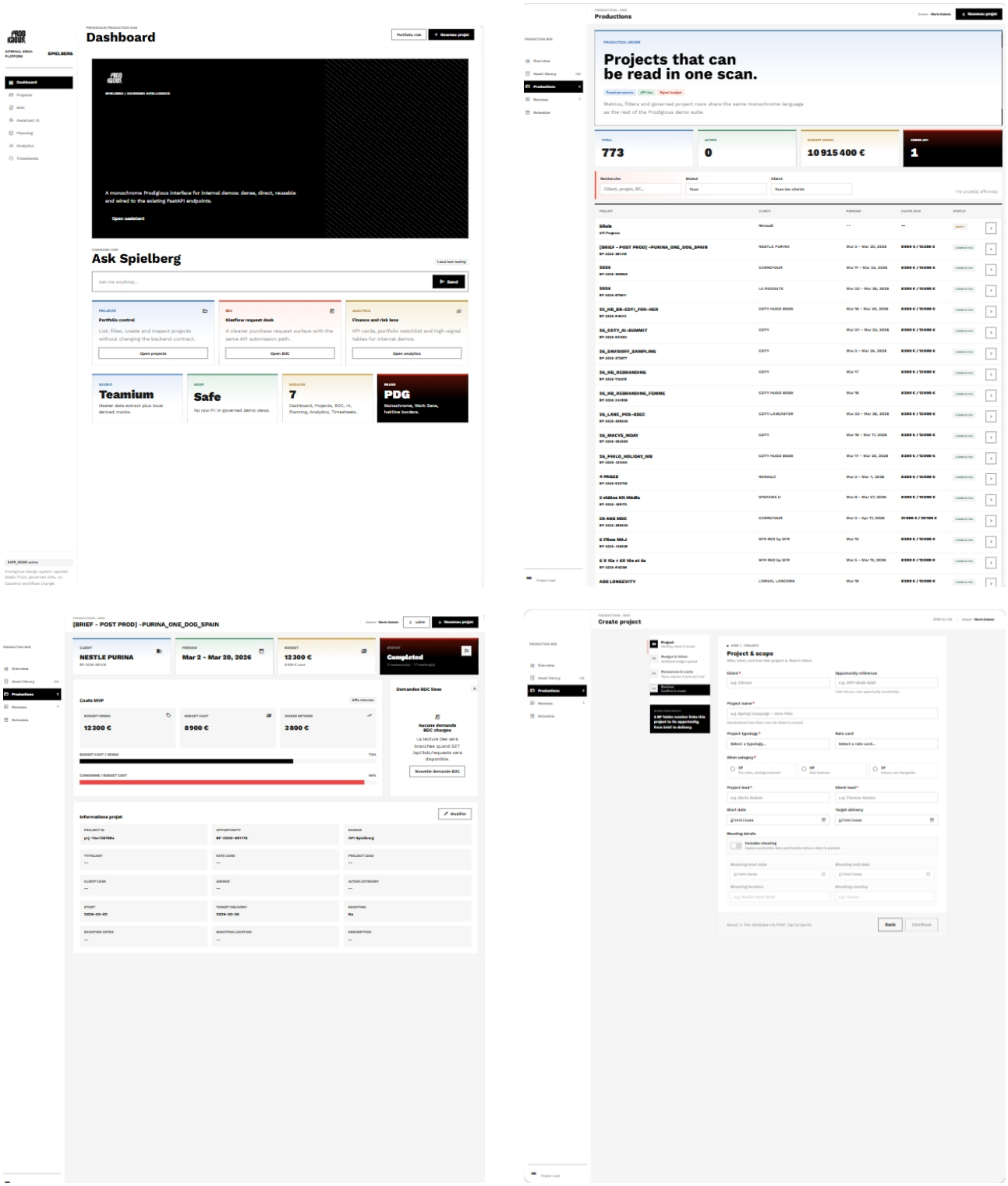


FIGURE 3.3 – Adaptation du design system aux interfaces développées : dashboard, liste des projets, chatbot et détail projet.

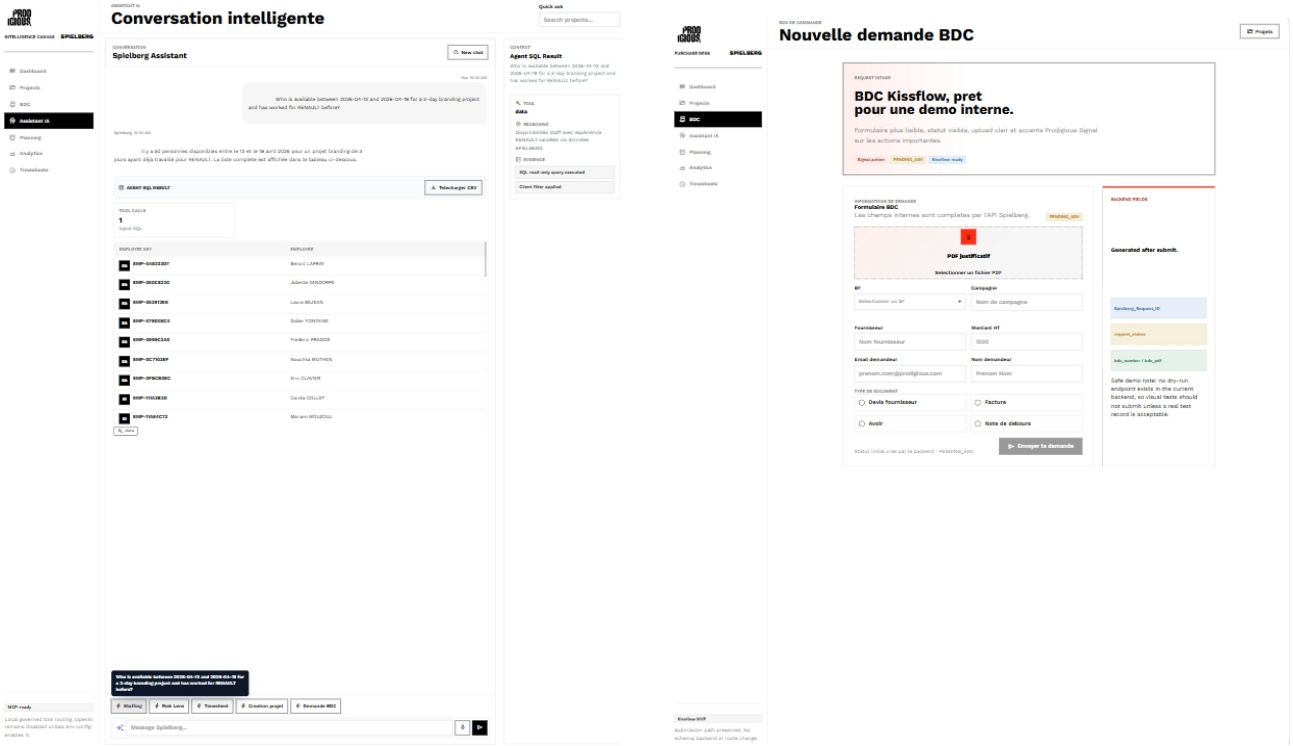


FIGURE 3.4 – Adaptation du design system pour les pages Dashboard, assistant IA, liste et détail des projets, et formulaires métier : demande BDC et création de projet.

Phase 6 – Interface de création projet

Principe technique :

La création d'un projet est un workflow métier structuré qui nécessite de collecter plusieurs informations hétérogènes : client, nom du projet, référence d'opportunité, typologie, catégorie Altair, dates, budget, statut, hypothèses de ressources et éventuelles informations de tournage. Pour éviter un formulaire trop dense, l'interface utilise un *wizard* qui découpe la saisie en étapes logiques. Cette approche permet de guider l'utilisateur, de limiter les erreurs de saisie et de valider progressivement les informations avant l'enregistrement.

Application dans Spielberg :

Dans Spielberg, le formulaire de création projet a été structuré en quatre étapes : *Project & scope*, *Budget & Altair*, *Resources & costs* et *Review*. Le frontend lit les champs saisis, construit un payload JSON, applique une première validation côté interface puis envoie la demande au backend via l'endpoint `POST /api/projects`. La validation côté frontend vérifie notamment les champs obligatoires, la cohérence des dates, les montants positifs et les champs de tournage lorsque l'option correspondante est activée.

Côté backend, la requête est validée avec le schéma Pydantic `ProjectCreateRequest`. Ce schéma contrôle les types, les champs obligatoires, les devises autorisées, les montants et la cohérence des dates. Le service `create_project_from_request()` transforme ensuite le modèle validé en données persistables : il génère un `project_id` stable à partir du client et du nom du projet avec `make_project_id()`, définit certaines valeurs par défaut comme `source=frontend` et `risk_level=low`, puis vérifie qu'un projet identique n'existe pas déjà. Enfin, le repository `create_project()` insère le projet dans la table `altair_projects` de la base SQLite locale.

Une fois créé, le projet devient visible dans la liste des projets, dans la vue détail et dans les tableaux de bord qui consomment les données exposées par les endpoints FastAPI. Le formulaire peut aussi être prérempli à partir d'un brouillon préparé par l'assistant, mais la validation finale et la création effective restent toujours déclenchées par l'utilisateur depuis l'interface.

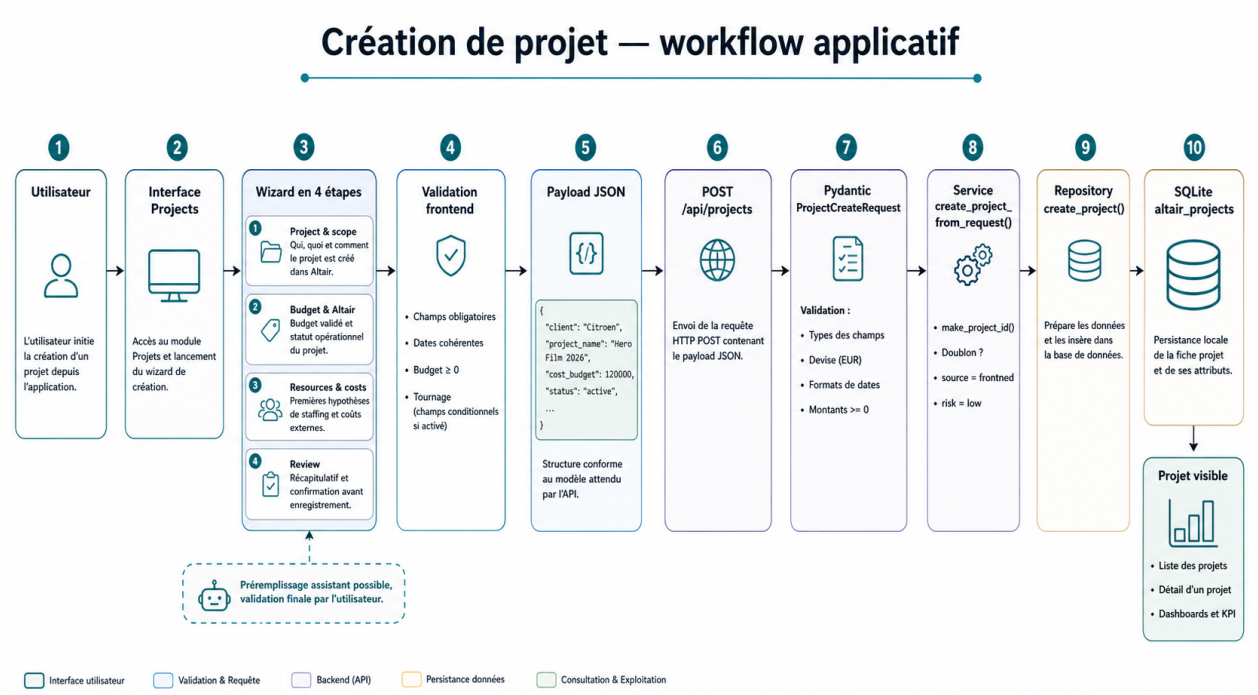


FIGURE 3.5 – Workflow applicatif de création d'un projet dans Spielberg.

Phase 7 – Interface BDC, API Kissflow et persistance locale

Principe technique :

Une demande de BDC correspond à un workflow métier sensible : elle contient des informations financières, un fournisseur, un demandeur, un rattachement projet ou BF, puis doit être transmise à un outil de workflow externe. L'architecture retenue sépare donc clairement la saisie utilisateur, la validation backend, le mapping vers l'API externe, l'appel Kissflow et la persistance locale. Cette séparation permet de limiter les erreurs, de tracer les demandes envoyées et de conserver une preuve côté Spielberg même si le workflow est ensuite traité dans Kissflow.

Application dans Spielberg :

Dans Spielberg, l'interface BDC permet à l'utilisateur de créer une demande à partir d'un formulaire dédié. Le formulaire collecte les champs nécessaires : `bf_code`, `campaign_name`, `document_type`, `supplier_name`, `amount_ht`, `requester_email` et `requester_name`. Le type de document est limité à des valeurs métier contrôlées, comme *Devis fournisseur*, *Facture*, *Avoir* ou *Note de débours*. Le frontend effectue une première validation puis envoie le payload au backend via l'endpoint `POST /api/bdc/requests`.

Côté backend, la requête est validée par le schéma Pydantic `BDCFrontCreateRequest`. Ce schéma vérifie notamment les champs obligatoires, le montant strictement positif, le format de l'email demandeur et la validité du type de document. Une normalisation est également appliquée sur le type de document afin d'accepter certaines variantes textuelles tout en les ramenant vers une valeur métier

standardisée.

Le service `create_submit_bdc_request()` transforme ensuite la demande frontend en modèle interne `BDCCreateRequest`. À cette étape, Spielberg génère un identifiant interne unique avec `generate_spielberg_r` au format `SPB-BDC-...`, puis initialise le statut de la demande à `PENDING_ADV`. Le mapper `map_bdc_create_to_kis` convertit ensuite le modèle interne en payload compatible avec Kissflow.

L'intégration Kissflow est isolée dans le client `KissflowClient`. Celui-ci construit les points d'entrée API à partir de la configuration d'environnement, ajoute les headers d'authentification requis, puis exécute deux appels principaux conformément au fonctionnement de l'API Kissflow [21]. Le premier appel crée l'item dans le process Kissflow via `create_item()`, puis le second soumet cet item dans le workflow via `submit_item()` en utilisant les identifiants techniques retournés par Kissflow.

Enfin, Spielberg conserve une trace locale de la demande dans SQLite. La table `bdc_requests` est créée par `init_db_schema()` si elle n'existe pas déjà. La fonction `insert_bdc_request()` y enregistre l'identifiant Spielberg, les champs métier de la demande, le statut, ainsi que les identifiants Kissflow `kissflow_id` et `kissflow_activity_instance_id`. Cette persistance locale permet de garder un historique côté Spielberg, de faciliter les audits et de relier une demande frontend à son exécution dans Kissflow.

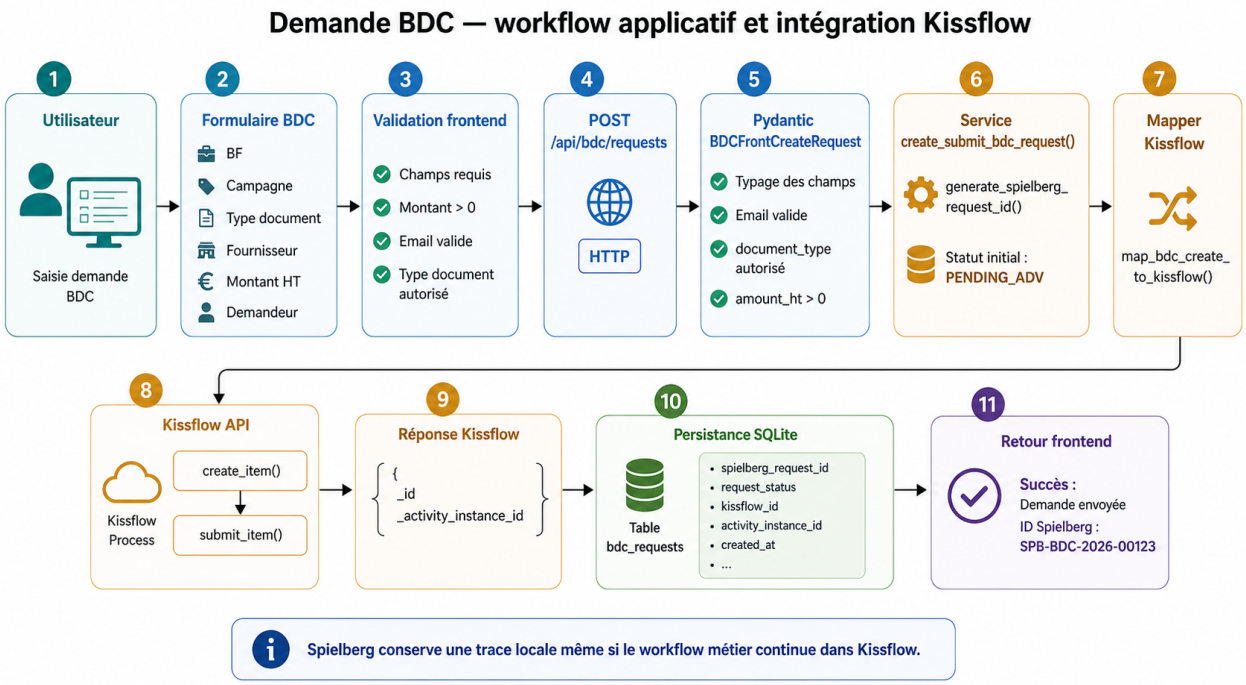


FIGURE 3.6 – Workflow applicatif de création et de soumission d'une demande BDC vers Kissflow.

Axe 2 – Implémentation des agents IA, architecture agentique et évaluation

Cet axe constitue la partie centrale du stage du point de vue de l'ingénierie IA. L'objectif n'était pas seulement d'ajouter un chatbot au POC, mais de faire évoluer progressivement une interface conversationnelle rigide, dans laquelle le LLM servait surtout à reformuler la demande utilisateur, vers une couche agentique où le LLM devient un composant central de décision. Cette couche permet de comprendre des demandes métier, de sélectionner l'outil approprié, de retourner des données structurées au frontend et de garder le contrôle sur les actions sensibles.

Définition générale : Un **agent IA** est un système logiciel dans lequel le LLM joue le rôle de moteur de raisonnement : il interprète la demande, comprend le contexte, choisit une stratégie et décide si un outil doit être appelé. Autour de ce noyau, l'agent s'appuie sur une mémoire conversationnelle, un ensemble d'outils contrôlés, des instructions système et parfois des sources documentaires externes. La réponse finale n'est donc pas seulement générée à partir du modèle, mais construite à partir d'une boucle entre raisonnement, contexte, appel d'outil et résultat observé.

Dans le projet Spielberg, cette logique permet au LLM de servir de couche de décision au-dessus de capacités déterministes : requêtes SQL en lecture seule, génération de brouillons de projet, préparation de demandes BDC et recherche documentaire. L'architecture agentique devient pertinente lorsque l'entrée utilisateur est ambiguë, conversationnelle ou multi-domaine, et lorsqu'il faut sélectionner dynamiquement la bonne capacité parmi plusieurs outils disponibles. [5, 7, 11].

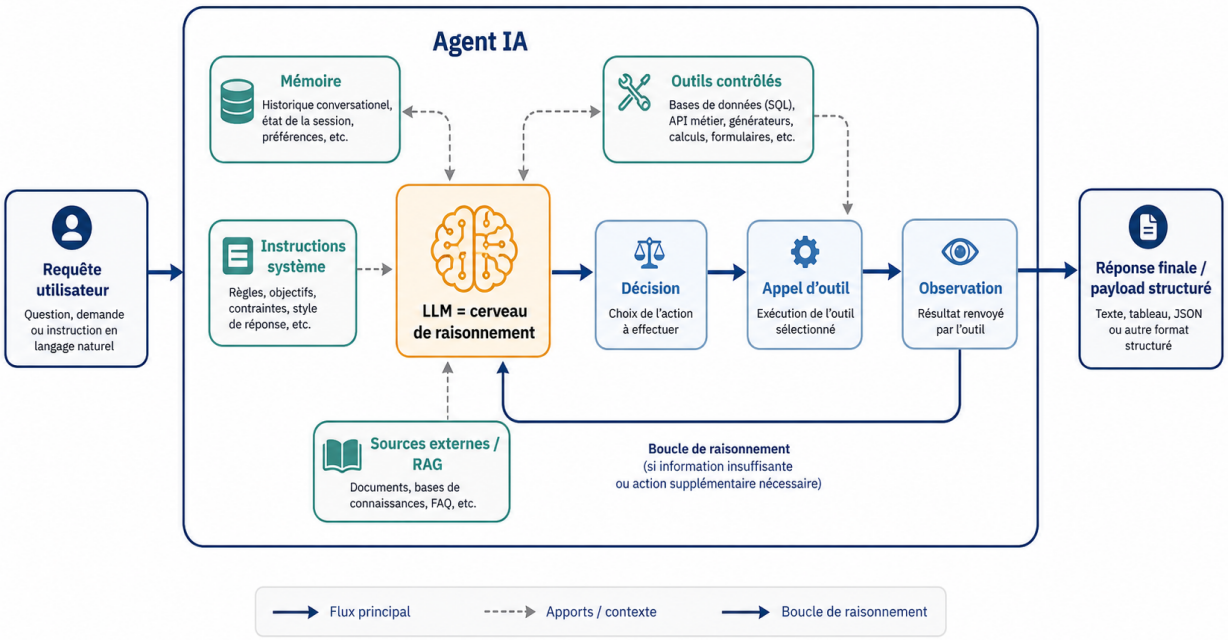


FIGURE 3.7 – Schéma général d'un agent IA : requête utilisateur, LLM, décision, appel d'outil, résultat structuré et réponse frontend

Dans Spielberg, l'utilisation de l'architecture agentique plus précisément multi-agentique se justifie par la diversité des usages : consultation de données, création projet, demande BDC, questions documentaires et suivi conversationnel. Ces usages n'ont pas les mêmes règles métier, les mêmes risques, ni les mêmes sorties frontend. Les réponses `data` doivent privilégier les tableaux structurés, les réponses `project` et `bdc` doivent produire des brouillons validables par l'utilisateur, et le RAG doit citer les sources documentaires sans inventer le contenu. L'enjeu principal est donc moins de rendre l'agent autonome que de contrôler son périmètre, son contexte et ses effets.

Architectures étudiées :

Plusieurs architectures agentiques ont été comparées avant de retenir le pattern Supervisor. L'agent unique avec tous les outils était le choix le plus naturel pour démarrer : il est simple, rapide à prototyper et suffisant pour valider un premier accès aux données. Ses limites apparaissent cependant dès que les domaines se multiplient : le prompt devient trop chargé, les règles SQL, projet, BDC et RAG se mélangent, le contexte coûte plus cher et les sorties frontend deviennent plus difficiles à stabiliser.

Le Router représentait une première amélioration, car il séparait les intentions et déléguait la demande au bon agent spécialisé. Cette approche reste claire pour un POC, mais elle devient fragile en conversation multi-tours : le router sait classer une demande, mais il ne possède pas vraiment l'état global de la conversation. Les architectures de type swarm ont aussi été considérées, mais elles sont moins adaptées à Spielberg car l'utilisateur peut changer de sujet pendant un formulaire projet ou BDC. Le pattern Subagents / Supervisor s'est finalement imposé car il conserve un interlocuteur unique, isole les workers spécialisés et garde le contrôle sur les actions sensibles.

TABLE 3.2 – Comparaison des architectures agentiques étudiées pour Spielberg.

Architecture	**Principe**	**Intérêt**	**Limite observée**
Agent unique	Un seul agent reçoit la conversation et choisit directement les outils.	Simple pour démarrer et valider un premier prototype.	Prompt chargé, contexte coûteux, mélange des règles data, projet, BDC et RAG.
Router	Classification de l'intention puis délégation vers un agent spécialisé.	Séparation claire des domaines pour un POC.	Reprise de contexte multi-tours fragile, car le router classe plus qu'il ne pilote la conversation.
Swarm	Un agent transfère la main à un autre agent qui poursuit le dialogue.	Pertinent pour des workflows séquentiels bien bornés.	Moins adapté si l'utilisateur change de sujet pendant un formulaire.
Supervisor	Un agent central garde le dialogue et appelle des workers spécialisés.	Contrôle centralisé, isolation du contexte, meilleure maîtrise des actions sensibles.	Demande de limiter les reformulations inutiles pour éviter tokens et latence.

Les documents LangChain distinguent plusieurs patterns multi-agents : subagents, swarms, skills, router et workflows personnalisés avec LangGraph [7, 8, 9]. Le point central est le *context engineering* : choisir ce que chaque agent doit voir, afin de limiter la surcharge de contexte et les erreurs de décision. Pour Spielberg, cette idée est déterminante : le worker `data` n'a pas besoin de voir tout le brouillon BDC, le worker `bdc` n'a pas besoin de recevoir les résultats SQL détaillés, et le worker `rag` doit surtout recevoir la question documentaire et les passages récupérés.

Critères de choix du Supervisor :

Le choix du Supervisor vient directement des besoins observés dans Spielberg. Les domaines métier sont distincts : consultation de données, création projet, demande BDC et documentation interne n'ont pas les mêmes règles ni les mêmes sorties. Les actions sensibles exigent aussi une séparation claire entre compréhension par le LLM, validation Pydantic, action backend et confirmation utilisateur. La conversation multi-tours renforce ce besoin : l'utilisateur peut commencer un brouillon projet, ajouter un budget plus tard, puis poser entre-temps une question data. Le supervisor reste alors responsable du fil conversationnel et de l'état actif.

Ce choix répond également aux contraintes frontend et aux enjeux de coût. Les workers doivent retourner des payloads structurés, par exemple un tableau, une carte, un formulaire prérempli ou des sources RAG, sans forcer le LLM à tout reformuler. Lorsque le worker produit déjà une sortie exploitable, Spielberg privilégie donc une finalisation déterministe afin de réduire les tokens, la latence et les risques de paraphrase. LangGraph apporte enfin un cadre plus industriel pour représenter l'état, les transitions, les points d'arrêt, les traces et une future logique de checkpointing.

Lecture des comparaisons LangChain :

Les comparaisons LangChain montrent que le choix d'une architecture multi-agent ne se limite pas au nombre d'agents. Une requête simple peut être traitée avec peu d'appels modèle par plusieurs patterns, mais les écarts apparaissent lorsque la conversation se répète ou devient multi-domaine. Les subagents gardent une bonne isolation de contexte, les swarms peuvent devenir moins adaptés si l'utilisateur change de sujet pendant un workflow, les skills sont utiles pour charger des connaissances ponctuelles mais peuvent accumuler du contexte, et le router reste efficace pour classifier sans suffire à gérer l'état conversationnel complet. Pour Spielberg, cette lecture a renforcé le choix d'un Supervisor contrôlé, avec des workers spécialisés et une finalisation déterministe dès que le résultat worker est déjà exploitable.

TABLE 3.3 – Lecture comparative des patterns multi-agents LangChain pour Spielberg.

Pattern	**Comportement observé**	**Lecture pour Spielberg**
Subagents	Bonne isolation de contexte, mais appels supplémentaires possibles.	Adapté au besoin de workers spécialisés appelés par un supervisor.
Swarm	Transfert direct de la conversation entre agents.	Moins adapté si l'utilisateur change de sujet pendant un workflow métier.
Skills	Chargement ciblé de capacités ou connaissances.	Utile ponctuellement, mais risque d'accumuler trop de contexte dans un assistant large.
Router	Classification efficace d'une demande isolée.	Bon point de départ, mais insuffisant seul pour gérer l'état multi-tours.

Les benchmarks LangChain montrent également que l'agent unique peut voir son coût en tokens augmenter lorsque plusieurs domaines distracteurs sont ajoutés, alors que les architectures multi-agents gardent mieux l'isolation du contexte [10].

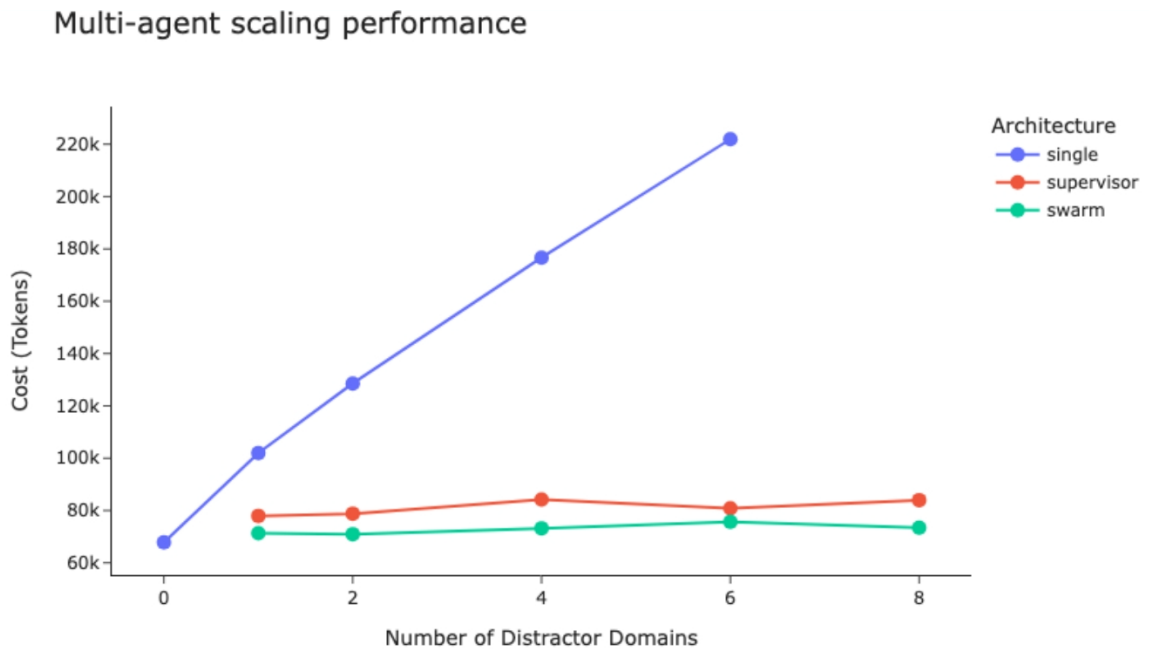


FIGURE 3.8 – Comparaison du coût en tokens entre un agent unique et des architectures multi-agents (Supervisor et swarm) et lorsque le nombre de domaines distracteurs augmente.

Ces analyses signalent cependant un risque important du pattern supervisor naïf : si le supervisor reformule systématiquement la réponse du worker, il ajoute une couche de traduction qui peut dégrader l'information, augmenter la latence et consommer davantage de tokens. Ce point a orienté le refactoring de l'architecture Supervisor dans Spielberg : lorsque le worker retourne déjà un contrat exploitable par le frontend, le graphe privilégie une finalisation déterministe. Ce choix évite aussi certaines boucles LLM-LLM susceptibles d'augmenter inutilement le nombre d'appels API en cas d'erreur ou d'hallucination.

Cette décision reprend le principe formalisé dans la documentation interne de refactoring : le supervisor parle, les workers exécutent, et la couche déterministe conserve le contrat de sortie attendu par le frontend [20].

Décision d'architecture retenue :

La couche IA de Spielberg repose donc sur un Supervisor comme interlocuteur conversationnel unique. Il décide s'il doit répondre directement ou appeler un worker spécialisé : `data`, `project`, `bdc` ou `rag`. Les workers isolent leurs prompts, leurs règles métier et leurs outils, ce qui évite de faire porter tout le raisonnement par un agent généraliste. LangGraph structure le flux autour de nœuds explicites, notamment `input_guard`, `supervisor_decision`, `run_worker` et `finalize_from_worker`. Enfin, l'évaluation LangSmith permet de mesurer la stabilité technique, le routage, la qualité des sorties structurées, les tokens et la latence par domaine.

Phase 8 – Du chat déterministe vers un premier agent LangChain

Principe technique :

Avant l'introduction des agents, le chatbot reposait sur une logique principalement déterministe. La fonction `answer_local()` analysait la requête utilisateur avec des règles explicites : extraction du rôle, du département, du client, des dates, du `project_id`, du `bf_id`, de l'`employee_key` ou encore du nombre minimal de jours disponibles. Ces éléments étaient extraits avec des expressions régulières et des listes de mots-clés, puis utilisés pour choisir manuellement le bon bloc de traitement : disponibilité, budget, timesheet, risque client, planning, ressources, projets ou analytics.

Cette première approche avait un avantage important pour un POC : elle était prévisible, contrôlée et compatible avec des payloads frontend bien structurés. Chaque branche construisait une réponse avec des `cards`, une `table`, des données brutes et parfois des éléments d'*evidence*. Même lorsque l'API OpenAI était utilisée dans `answer_with_openai()`, le LLM ne décidait pas réellement de l'action à exécuter : le backend calculait d'abord une sortie sécurisée avec `answer_local()`, puis le modèle servait surtout à reformuler une réponse concise à partir de ce résultat pseudonymisé.

Cependant, cette architecture devenait difficile à faire évoluer. Chaque nouveau cas d'usage demandait d'ajouter des mots-clés, des conditions, des extracteurs ou une nouvelle branche dans `answer_local()`. Le système pouvait donc répondre correctement à des formulations prévues, mais restait fragile face à des demandes plus naturelles, ambiguës ou multi-intentions.

Évolution par rapport au chat déterministe :

Le passage au premier agent LangChain a déplacé la compréhension utilisateur depuis des règles regex et des mots-clés vers le raisonnement du LLM et la sélection dynamique d'outil. Cette comparaison résume le changement de logique :

TABLE 3.4 – Comparaison entre le chat déterministe initial et le premier agent LangChain.

Aspect	**Chat déterministe**	**Premier agent LangChain**
Compréhension utilisateur	Regex, mots-clés et conditions explicites.	Raisonnement du LLM et sélection dynamique d'outil.
Ajout d'un cas d'usage	Nouvelle branche de code ou nouvel extracteur.	Extension possible par prompt et tools disponibles.
Accès aux données	Appels directs à des fonctions métier prévues.	Tool SQL contrôlé, limité à la lecture seule.
Robustesse	Très stable sur les cas prévus, fragile sur les formulations imprévues.	Plus flexible linguistiquement, mais nécessite davantage de garde-fous.
Payload frontend	Construit manuellement et donc très maîtrisé.	Reconstruit à partir des appels tools et de leurs résultats.

En contrepartie, l'agent demande plus de contrôle : les réponses doivent rester structurées et les actions sensibles ne peuvent pas être laissées à une simple génération de texte.

Application dans Spielberg :

L'idée naturelle après cette première version était de créer un **agent unique généraliste**. Cette étape correspond au passage vers `agent_service.py`, où un agent LangChain est créé avec `create_agent()`, un modèle `ChatOpenAI`, un prompt système métier et un premier tool `sql_query`. Ce tool permet à l'agent d'interroger la base SQLite locale, tout en restant protégé par une règle importante : seules les requêtes `SELECT`, `WITH ... SELECT` et `PRAGMA` sont autorisées. Les opérations d'écriture comme `INSERT`, `UPDATE`, `DELETE`, `DROP` ou `ALTER` sont bloquées.

Ce premier agent a donc déplacé une partie de l'intelligence depuis le code déterministe vers le LLM. Au lieu de maintenir une grande liste de conditions pour chaque formulation possible, l'agent pouvait interpréter une question plus libre, décider d'appeler le tool SQL, générer une requête adaptée, puis retourner un résultat exploitable. Le backend reconstruisait ensuite une réponse structurée avec `cards`, `table`, `tool_calls`, `tool_results` et `evidence`.

Cette étape a amélioré la souplesse de l'assistant pour les questions data. Elle a aussi mis en évidence les limites du single agent : prompt de plus en plus long, risque d'erreur SQL, difficulté à mélanger consultation de données, création projet, demande BDC et RAG dans un seul espace de décision, ainsi que fragilité des sorties structurées attendues par le frontend. Ces limites ont motivé la phase suivante : passer d'un agent unique à une architecture orchestrée, puis à un supervisor avec workers spécialisés.

Use cases couverts à cette étape :

Les premiers cas traités concernaient principalement la consultation de données : recherche de projets, ressources, disponibilités, planning, budgets, analytics et questions générales sur le périmètre Spielberg. Les cas plus sensibles, comme la création de projet ou la demande BDC, ont ensuite nécessité une séparation plus stricte entre compréhension LLM, validation backend et action métier.

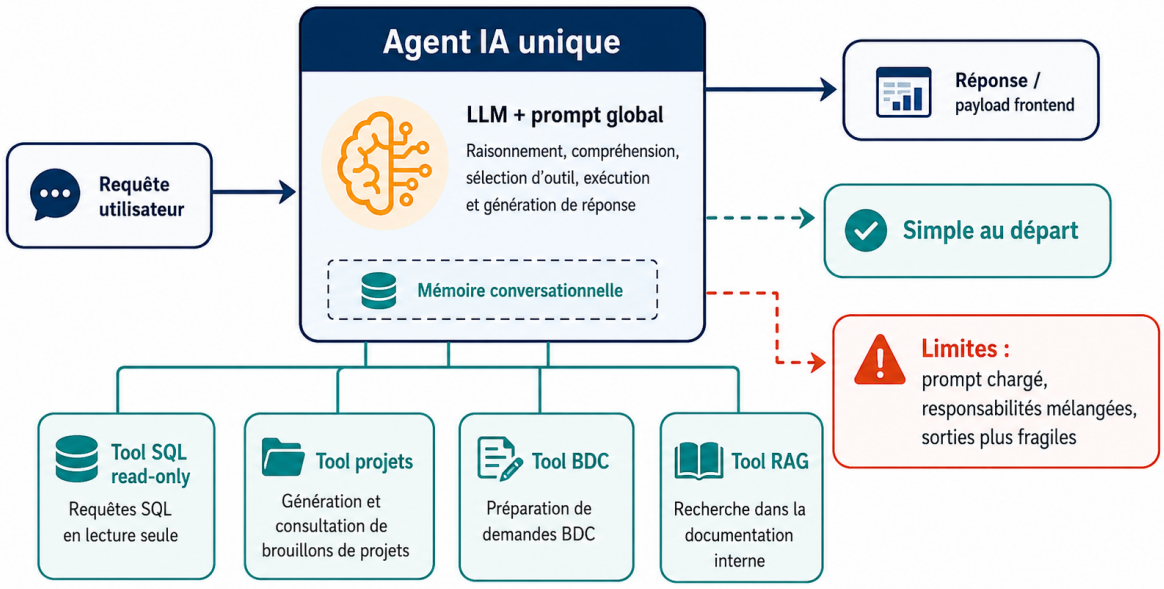


FIGURE 3.9 – Architecture du premier agent unique : un seul agent LangChain centralise le raisonnement, le prompt global, la mémoire conversationnelle et l'accès direct aux outils.

Transition vers LangGraph et les architectures multi-agents :

Après le premier agent LangChain, la suite logique a été de réfléchir à une architecture capable de dépasser le modèle d'un agent unique. Le besoin n'était plus seulement d'appeler un outil SQL, mais de faire cohabiter plusieurs capacités : lecture de données, création de projet, préparation de BDC et recherche documentaire. Cette évolution impose de gérer l'état de la conversation, les transitions entre étapes, les retours des outils, les erreurs et les traces d'exécution.

LangGraph s'est imposé comme une solution adaptée car il permet de représenter l'assistant non plus comme une simple fonction appelée à chaque message, mais comme un **graphe d'état**. Chaque nœud peut correspondre à une étape claire : analyse de la demande, routage, appel d'un agent spécialisé, exécution d'un outil, finalisation de la réponse ou gestion d'une erreur. Cette représentation rend l'architecture plus lisible, plus testable et plus facilement observable [12].

Un autre intérêt de LangGraph est la possibilité de garder des traces de l'exécution : quel nœud a été appelé, quel agent a répondu, quel outil a été utilisé, quel état conversationnel était actif et quelle sortie structurée a été retournée. Cette traçabilité prépare directement l'évaluation avec LangSmith, utilisée ensuite pour mesurer les performances du supervisor : taux de succès, qualité fonctionnelle, latence, tokens consommés et stabilité par domaine.

Cette réflexion a conduit à une première architecture multi-agent basée sur un **Router**. Le router représentait une étape intermédiaire naturelle : au lieu de donner tous les outils à un seul agent, il classifie l'intention de l'utilisateur et redirige la demande vers l'agent spécialisé le plus adapté.

Phase 9 – Orchestrateur Router : première architecture multi-agent

Principe technique :

Le pattern *Router* consiste à placer un composant de classification en amont des agents spécialisés. À chaque message utilisateur, le router analyse l'intention et décide vers quel agent ou handler envoyer

la requête. Cette architecture est plus structurée qu'un agent unique, car elle sépare les responsabilités : un agent pour les questions data, un agent pour la création de projet, un agent pour les demandes BDC, et éventuellement un agent général pour les demandes non couvertes.

Dans Spielberg, cette approche représentait une suite logique après le premier agent LangChain. Le single agent devenait difficile à maintenir car il devait gérer trop de règles, trop d'outils et trop de formats de sortie. Le router permettait donc de découper le problème en domaines fonctionnels plus clairs.

Application dans Spielberg :

L'orchestrateur Router a été introduit pour orienter les messages utilisateur vers différents agents spécialisés : `data_agent` pour les questions de consultation, disponibilité, planning, projets, budgets et analytics, `create_project` pour la préparation d'un brouillon de création de projet, `create_bdc` pour la préparation d'une demande de BDC, et un fallback général lorsque l'intention était trop vague ou hors périmètre.

Cette étape a amélioré la lisibilité de l'architecture. Chaque agent pouvait avoir son propre prompt, ses propres règles métier et ses propres sorties structurées. Le router évitait aussi de donner tous les outils à un seul agent, ce qui réduisait la surcharge du prompt global.

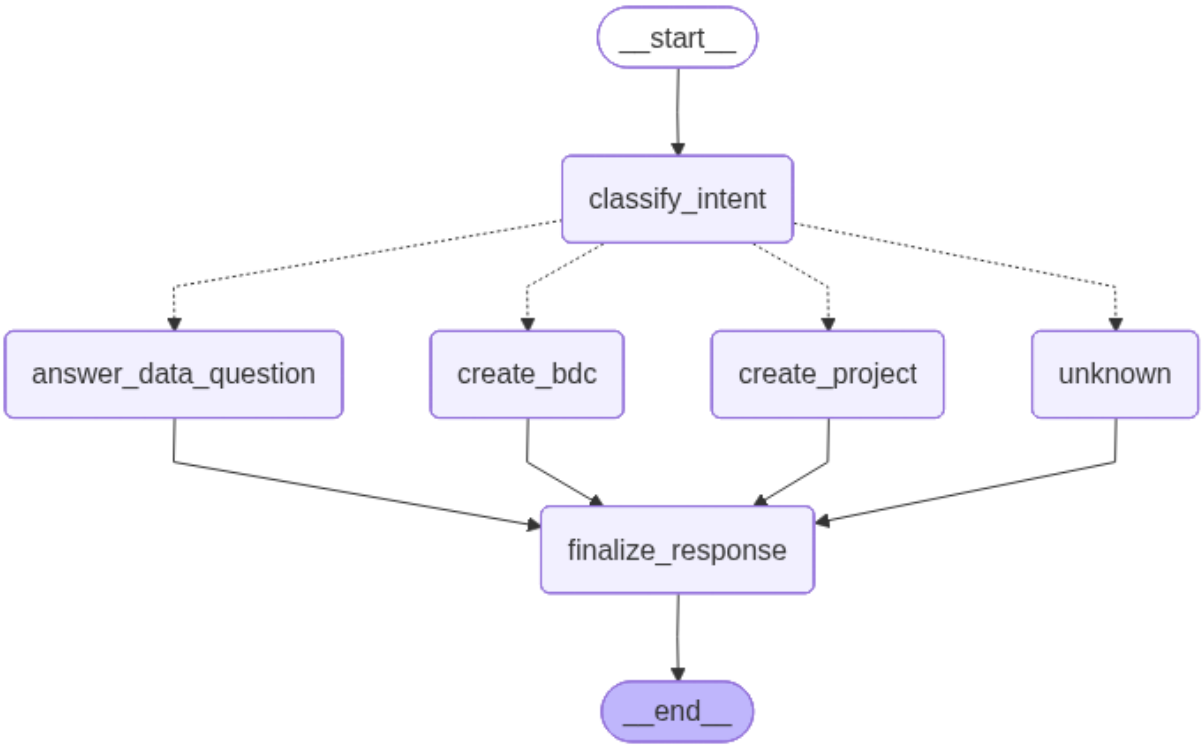


FIGURE 3.10 – Première architecture multi-agent basée sur un Router : classification de l'intention puis délégation vers un agent spécialisé.

Limites rencontrées : Cette architecture a cependant montré ses limites dès que les conversations devenaient multi-tours. Le router fonctionnait bien pour une demande isolée, mais devenait plus fragile lorsque l'utilisateur poursuivait un formulaire déjà commencé ou changeait temporairement de sujet.

Par exemple, lorsqu'un formulaire de création projet ou BDC était en cours, une règle rigide consistait à laisser parler l'agent spécialisé concerné. Cette logique fonctionnait tant que l'utilisateur conti-

nuait exactement le workflow attendu. Mais si l'utilisateur donnait une information partielle, posait une question data au milieu du formulaire, ou revenait ensuite au brouillon, le router pouvait perdre le contexte ou router vers le mauvais agent.

Le problème principal était donc la responsabilité conversationnelle. L'utilisateur n'avait pas réellement l'impression de parler à un seul assistant cohérent, mais plutôt à plusieurs agents qui se passaient la main. Chaque agent pouvait gérer son domaine, mais aucun composant ne possédait vraiment toute la conversation, les brouillons actifs, les actions en attente et les changements d'intention.

Transition vers le Supervisor C'est à partir de cette limite qu'est née l'idée d'une architecture où un agent central garderait le contrôle du dialogue, tandis que les agents spécialisés deviendraient des workers appelés uniquement lorsque nécessaire. L'idée initiale était donc pragmatique : garder un seul interlocuteur pour l'utilisateur, mais conserver des spécialistes derrière lui.

En approfondissant les architectures multi-agents, cette idée s'est révélée correspondre au pattern *Supervisor*, déjà documenté dans l'écosystème LangChain [7, 9]. Le Supervisor répond précisément au problème rencontré : il conserve le contexte global, décide à chaque tour si un worker doit être appelé, et évite que l'utilisateur dialogue directement avec plusieurs agents concurrents.

Phase 10 – Architecture Supervisor LangGraph et workers spécialisés

Principe technique :

Après les limites rencontrées avec le Router, l'évolution naturelle a été de passer à une architecture de type **Supervisor**. Dans ce pattern, l'utilisateur ne dialogue plus avec plusieurs agents spécialisés : il parle à un seul agent central, le supervisor. Les agents spécialisés deviennent des *workers*, appelés uniquement lorsque leur expertise est nécessaire.

Cette séparation répond directement aux problèmes observés avec l'orchestrateur précédent : perte de contexte, difficulté de suivi multi-tours, responsabilités floues entre agents et impression de parler à plusieurs assistants différents. Avec le Supervisor, un seul composant garde la responsabilité du dialogue, du routage, de l'état conversationnel, des brouillons actifs, des actions en attente et de la réponse finale.

L'idée centrale retenue est : Le supervisor parle. Les workers exécutent.

Application dans Spielberg :

Dans Spielberg, le supervisor est implémenté avec `LangGraph`. Le système n'est plus une simple fonction appelée à chaque message, mais un graphe d'état composé de nœuds explicites. Le fichier `supervisor/graph.py` définit quatre nœuds principaux : `input_guard`, `supervisor_decision`, `run_worker` et `finalize_from_worker`.

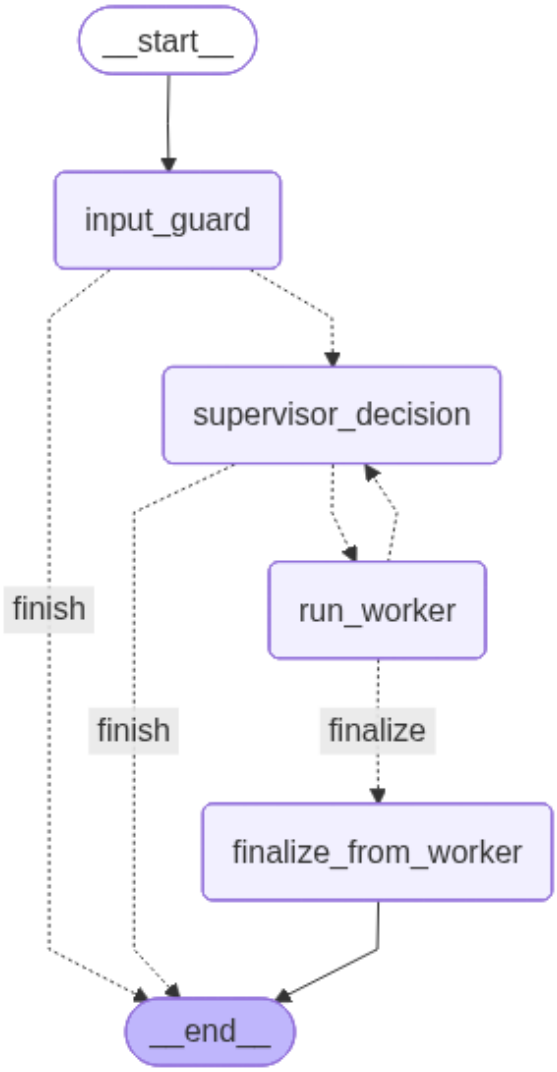


FIGURE 3.11 – Graphe LangGraph de l'architecture Supervisor Spielberg.

Le nœud `input_guard` applique des garde-fous déterministes avant l'appel au LLM. Il permet de bloquer les demandes hors périmètre, les tentatives de prompt injection, la révélation de secrets, les requêtes SQL brutes, les exports massifs, l'accès direct à la base de données et les demandes sensibles non autorisées. Ce choix est important car certaines règles de sécurité ne doivent pas dépendre uniquement du raisonnement probabiliste du modèle.

Le nœud `supervisor_decision` appelle le LLM supervisor et lui impose une sortie structurée `SupervisorDecision`. Cette décision indique si le système doit répondre directement ou appeler un worker. Elle contient également le worker choisi, un message reformulé pour ce worker, une réponse éventuelle et une justification courte. Le LLM garde donc un rôle de décision, mais cette décision reste encadrée par un schéma Pydantic.

Le nœud `run_worker` exécute le worker sélectionné parmi `data`, `project`, `bdc` et `rag`. Chaque worker reçoit uniquement le contexte utile à sa tâche : le message, la session, un éventuel brouillon, une action en attente et son contexte local. Cette isolation évite de surcharger chaque agent avec tout l'historique conversationnel.

Enfin, `finalize_from_worker` transforme le résultat du worker en réponse `SpielbergResponse` exploitable par le frontend. Une décision importante a été de ne pas repasser systématiquement par un second appel LLM après l'exécution d'un worker. Lorsque le worker retourne déjà un payload structuré, le système le transmet directement au frontend avec une synthèse courte. Cela réduit la latence, les coûts tokens, les risques de paraphrase et la perte de données structurées.

Workers spécialisés :

Le supervisor délègue quatre workers : `data` pour la consultation read-only des données de projets, staffing, planning, budgets, timesheets et disponibilités, `project` pour constituer un brouillon de projet et préparer un formulaire prérempli, `bdc` pour constituer un brouillon de demande BDC et préparer le formulaire correspondant, et `rag` pour rechercher et synthétiser les documents internes indexés.

Cette organisation permet de conserver un seul interlocuteur pour l'utilisateur tout en gardant des spécialistes derrière lui. Elle rend également l'architecture plus maintenable : chaque worker possède ses propres règles, son propre prompt et ses propres formats de sortie.

Garde-fous et actions sensibles : Les garde-fous ont été conçus à deux niveaux. Le premier niveau est déterministe, avec `input_guard`. Le second niveau est porté par le prompt supervisor : périmètre métier autorisé, refus hors scope, interdiction de révéler des secrets, interdiction de produire du SQL libre, et obligation de validation humaine pour les actions sensibles.

Dans Spielberg, les workers `project` et `bdc` ne créent pas directement un projet ou une demande depuis le chat. Ils préparent un brouillon et un formulaire prérempli, mais la validation finale reste effectuée par l'utilisateur dans l'interface. Cette séparation est essentielle pour garder le contrôle métier : le LLM comprend et prépare, mais l'exécution reste déterministe et validée.

Adaptation au frontend : Le supervisor a aussi été conçu en tenant compte de l'interface dans laquelle il vit. Lorsque le worker `data` retourne un tableau, le frontend l'affiche déjà sous la réponse ; le supervisor ne doit donc pas recopier toutes les lignes dans le texte. Il produit plutôt une synthèse courte, les critères appliqués et les points d'attention. De même, lorsque le worker `rag` retourne des sources, celles-ci sont affichées sous la réponse et n'ont pas besoin d'être relistées intégralement.

Cette règle améliore la lisibilité de l'assistant et limite la consommation de tokens. Elle évite aussi de dupliquer l'information entre la réponse textuelle et les composants frontend.

Justification : Cette architecture répond aux limites rencontrées avec le Router. Le Router classait correctement des intentions simples, mais il ne possédait pas vraiment la conversation. Le Supervisor, lui, garde le contexte global, les brouillons, les actions en attente et le contrôle du dialogue. Il permet donc de gérer les changements de sujet, les formulaires multi-tours et les retours de workers de manière plus robuste.

Cette phase marque ainsi le passage d'un assistant conversationnel expérimental à une architecture agentique plus structurée, plus maintenable et plus proche des patterns multi-agents modernes documentés par LangChain.

Phase 11 – Worker RAG documentaire intégré au Supervisor

Principe technique :

Le RAG, pour *Retrieval-Augmented Generation*, permet à un modèle de langage de répondre non seulement à partir de ses connaissances internes, mais aussi à partir de documents retrouvés dynamiquement dans une base documentaire. Dans un contexte métier, cette approche est importante car le LLM ne connaît pas forcément les documents internes, les procédures, les comptes rendus, les rapports ou les décisions récentes du projet.

Sans RAG, une réponse peut être modélisée par :

$$y = \arg\max_{y} P_\theta(y \mid q) \tag{3.1}$$

où q représente la question utilisateur et θ les paramètres internes du modèle. Avec le RAG, la génération dépend aussi des passages documentaires retrouvés :

$$y = \arg\max_{y} P_\theta(y \mid q, D_k(q)) \tag{3.2}$$

où $D_k(q)$ désigne les k passages les plus pertinents retrouvés pour la question q.

Dans Spielberg, le RAG n'est pas un module isolé : il est intégré comme un **worker spécialisé du Supervisor**. Le supervisor décide d'appeler le worker `rag` lorsque la question concerne la documentation interne, les guides, les procédures, les rapports ou les politiques indexées. Le worker RAG exécute alors la recherche documentaire, construit un contexte à partir des passages retrouvés, puis génère une réponse synthétique et sourcée.

Pipeline technique implémenté :

Le pipeline RAG est implémenté dans le module `rag/rag.py`. Il suit une chaîne simple mais complète :

$$\text{Documents} \rightarrow \text{Extraction} \rightarrow \text{Chunks} \rightarrow \text{Embeddings} \rightarrow \text{Chroma} \rightarrow \text{Retrieval} \rightarrow \text{Worker RAG} \tag{3.3}$$

Le module accepte plusieurs formats : PDF, TXT, Markdown et DOCX. Chaque fichier est transformé en blocs textuels avec des métadonnées comme la source, le nom du fichier, la page lorsque disponible et le type de document.

Le texte extrait est ensuite découpé en chunks à l'aide d'un découpage récursif. Dans l'implémentation actuelle, la taille de chunk est fixée à `800` caractères avec un chevauchement de `120` caractères entre deux fragments successifs.

Le chevauchement limite le risque de couper une information importante entre deux chunks consécutifs. On peut représenter la position de début d'un chunk par :

$$\text{start}(c_i) = i \times (s - o) \tag{3.4}$$

où s est la taille du chunk et o le chevauchement.

Chaque chunk est ensuite transformé en embedding avec le modèle défini par la variable `OPENAI_EMBEDDING_MODE` dont la valeur par défaut est `text-embedding-3-small`. On obtient donc, pour chaque chunk c_i, un vecteur :

$$e_i = f(c_i) \tag{3.5}$$

La question utilisateur est vectorisée de la même manière :

$$e_q = f(q) \tag{3.6}$$

La recherche documentaire consiste à retrouver les chunks les plus proches de la question, par similarité vectorielle. On peut l'exprimer ainsi :

$$D_k(q) = \text{TopK}_{c_i \in C} \, \text{sim}(f(q), f(c_i)) \tag{3.7}$$

Dans le projet, la recherche utilise `search_document(message, k=4)`. Le worker RAG récupère donc quatre passages pertinents au maximum, ce qui permet d'apporter des sources utiles sans surcharger le prompt envoyé au modèle.

Stockage vectoriel avec Chroma

Les embeddings, les chunks et leurs métadonnées sont stockés dans une base vectorielle locale Chroma. Le dossier de persistance est `backend/rag/rag_index/chroma`, et la collection utilisée est `spielberg_documents`. Ce choix est adapté au POC : Chroma fonctionne localement, persiste les données sur disque et s'intègre simplement avec LangChain.

Lors de l'indexation, la fonction `index_documents()` extrait les documents, les découpe, puis ajoute les chunks dans Chroma. Chaque chunk reçoit un identifiant stable construit à partir de la source, de la page, du `chunk_id` et d'un hash du contenu. Cela limite les collisions et permet de tracer plus facilement l'origine des passages.

Application dans le Supervisor :

Le worker RAG reçoit une `WorkerRequest` depuis le supervisor, contenant notamment le message utilisateur et l'identifiant de session. Son rôle est volontairement limité : il ne parle pas directement à l'utilisateur final, mais retourne un `WorkerResult` structuré au supervisor.

Le worker RAG récupère d'abord la question utilisateur, indexe les documents disponibles avec `index_documents()`, puis recherche les passages pertinents avec `search_document(message, k=4)`. Il construit ensuite une liste de sources avec `source`, `page`, `chunk_id` et un extrait, injecte ces passages dans le prompt du worker RAG, génère une réponse courte en français fondée uniquement sur les extraits fournis, puis retourne au supervisor une réponse structurée avec `cards`, `table`, `raw` et `evidence`.

Le worker retourne également un tableau de sources destiné au frontend. Chaque ligne contient le nom ou le chemin du document dans `source`, la page lorsque disponible, l'identifiant du fragment dans `chunk_id` et un extrait textuel limité dans `preview`.

Cette structure est importante car le frontend affiche déjà les sources sous la réponse. Le supervisor sait donc qu'il ne doit pas relister toutes les sources dans le texte final : il doit produire une synthèse courte, tandis que le tableau présente les quatre sources récupérées.

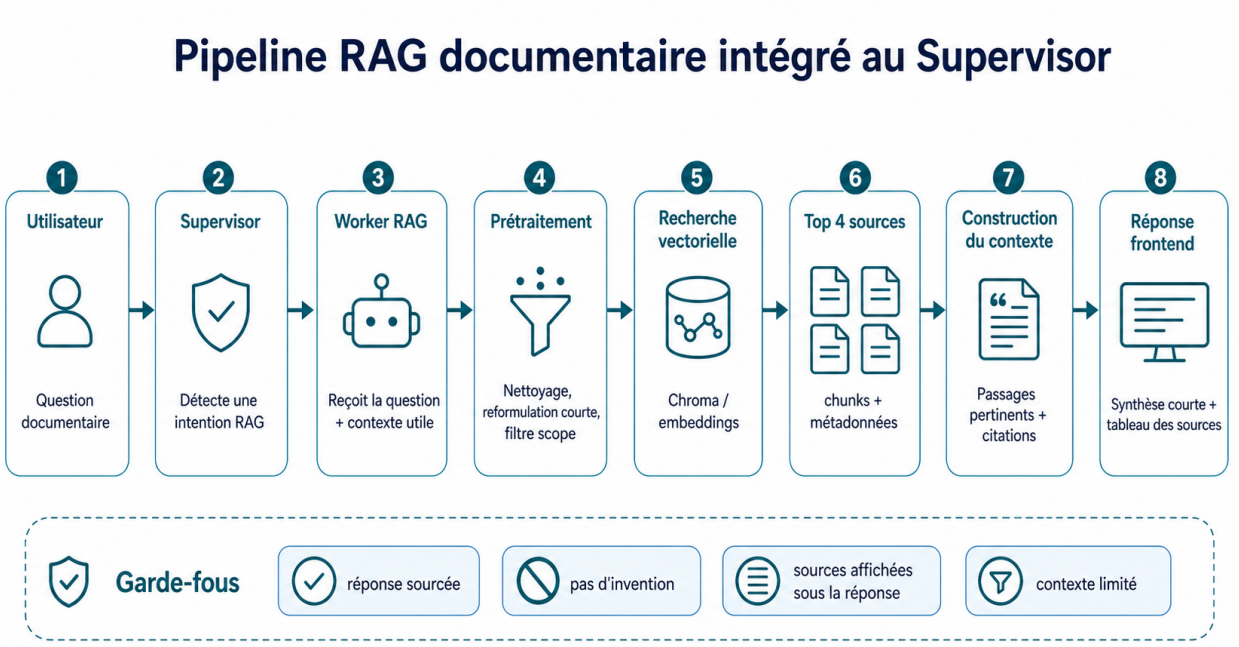


FIGURE 3.12 – Pipeline RAG documentaire intégré au Supervisor : détection de l'intention, appel du worker RAG, recherche vectorielle, sélection des sources et réponse structurée côté frontend.

Garde-fous du worker RAG : Le prompt du worker RAG impose plusieurs règles : utiliser uniquement les extraits documentaires fournis, ne jamais inventer une procédure, une politique, une source, une page ou une règle absente du contexte, indiquer clairement lorsque les documents récupérés ne permettent pas de répondre, citer les sources utiles avec le nom du document et la page lorsque disponible, et produire une synthèse concise au lieu de recopier tous les extraits.

Ces règles sont complémentaires aux garde-fous du Supervisor. Le supervisor décide si la question doit aller vers le RAG ; le worker RAG, lui, limite sa génération aux passages retrouvés. Cette séparation évite de mélanger les questions documentaires avec les questions data ou les workflows projet/BDC.

Résultat produit :

Lorsque des passages sont trouvés, le worker retourne une synthèse textuelle courte, une card indiquant le nombre de sources retrouvées, une autre card indiquant le nombre de chunks indexés, un tableau `RAG Sources` listant les sources, pages, chunks et previews, ainsi qu'un champ `evidence` reprenant les passages utilisés.

Si aucun passage pertinent n'est trouvé, le worker retourne une réponse contrôlée indiquant qu'aucun passage documentaire pertinent n'a été trouvé. Si une erreur survient pendant l'indexation ou la recherche, le worker retourne un statut `error` avec les documents disponibles et le message d'erreur dans `raw`.

Limites et perspectives : La version actuelle du RAG reste volontairement simple afin de valider le flux de bout en bout. Elle présente plusieurs limites : extraction PDF encore basique, pas d'OCR pour les documents scannés, absence de reranking, recherche uniquement vectorielle, et gestion documentaire des droits encore limitée.

Les améliorations possibles concernent l'extraction des PDF et des tableaux, l'ajout d'un OCR

pour les documents scannés, le test d'un reranker après la recherche vectorielle, la combinaison entre recherche vectorielle et recherche par mots-clés, l'évaluation automatique de la pertinence des passages retrouvés et de la fidélité des réponses aux sources, ainsi qu'une intégration plus fine des droits d'accès documentaires.

Cette phase apporte donc une brique essentielle à l'assistant Spielberg : la capacité de répondre à partir de la documentation interne, tout en restant dans l'architecture Supervisor et en conservant une traçabilité des sources affichable dans le frontend.

3.2.2 Résultats des expérimentations ou tests techniques

Les résultats expérimentaux combinent les tests applicatifs classiques et l'évaluation LangSmith de la couche agentique. Les vérifications portent principalement sur :

— la validation manuelle des endpoints FastAPI et la consultation de la base SQLite ;
— le flux BDC de création puis soumission avec Kissflow ;
— les scénarios assistant exécutés dans LangSmith ;
— le score fonctionnel structurel du supervisor et la qualité des sorties SQL structurées ;
— la latence moyenne, médiane et p95, ainsi que le coût estimé des exécutions.

Le score fonctionnel structurel correspond au nombre d'exécutions dont la sortie respecte le comportement attendu : bon routage, bon worker appelé, payload exploitable côté frontend et absence d'erreur bloquante.

Les 27 scénarios utilisés constituent un jeu d'évaluation représentatif des principaux parcours du POC, mais pas un échantillon statistique d'usage réel en production. Les résultats doivent donc être interprétés comme une mesure de validation technique et fonctionnelle du prototype.

Table 3.5 – Synthèse des résultats LangSmith du supervisor Spielberg.

Indicateur	**Résultat observé**
Scénarios évalués	27 scénarios répétés 2 fois
Exécutions LangSmith	54 exécutions
Succès techniques	54/54, soit 100 %
Score fonctionnel structurel	48/54, soit 88,9 %
Score textuel strict	45/54, soit 83,3 %
Sorties SQL avec table	22/22 sorties `agent_sql`
Erreurs SQL internes	0
Latence moyenne	4,53 s
Latence médiane	3,01 s
Latence p95	10,56 s
Latence maximale	71,52 s, outlier ponctuel
Tokens moyens	1 758 tokens par exécution
Coût total estimé	0,045331 USD sur 54 exécutions
Coût moyen par exécution	0,000839 USD

Stabilité technique : La première lecture concerne la stabilité globale. Le taux d'erreur des traces reste à 0 %, ce qui confirme les 54 exécutions en succès technique. Cette mesure ne signifie pas que toutes les réponses sont parfaites fonctionnellement, mais elle indique que le graphe supervisor, les workers et les outils ne provoquent pas d'erreur bloquante pendant l'évaluation.

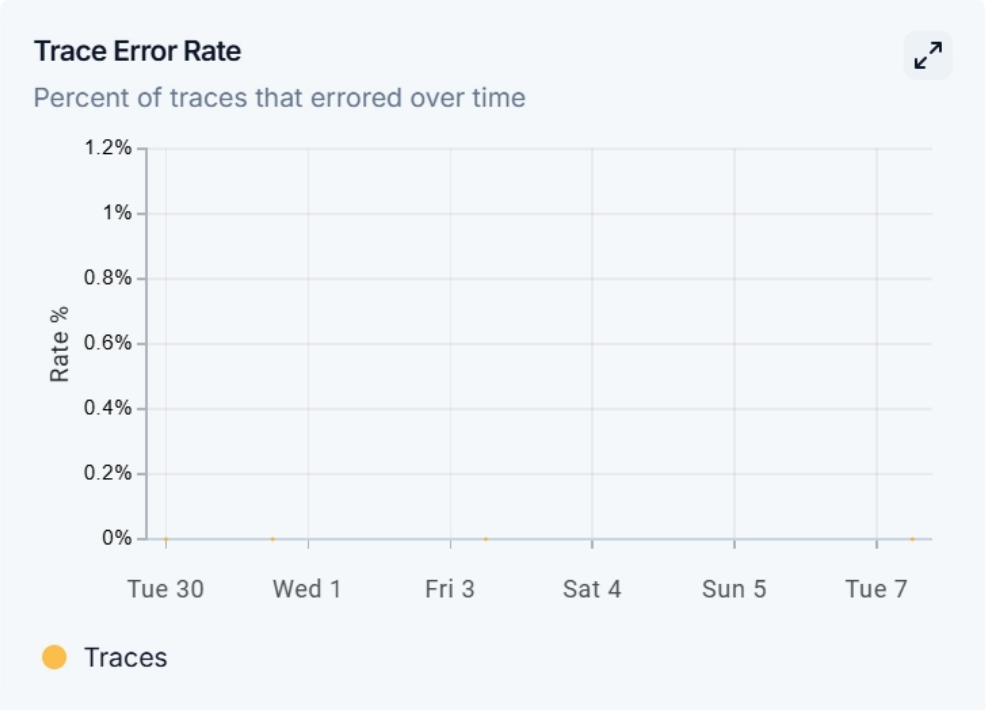


FIGURE 3.13 – Taux d'erreur des traces LangSmith pendant les évaluations du supervisor.

Analyse de la latence : La courbe de latence montre une amélioration des percentiles hauts au fil des runs. Les premiers essais présentent des P99 élevés, autour de 18 à 24 secondes, alors que le dernier point observé descend autour de quelques secondes. Cette évolution est cohérente avec le refactoring du supervisor : suppression de latences inutiles, réduction des boucles LLM et conservation directe des payloads structurés lorsque le worker produit déjà un résultat exploitable.

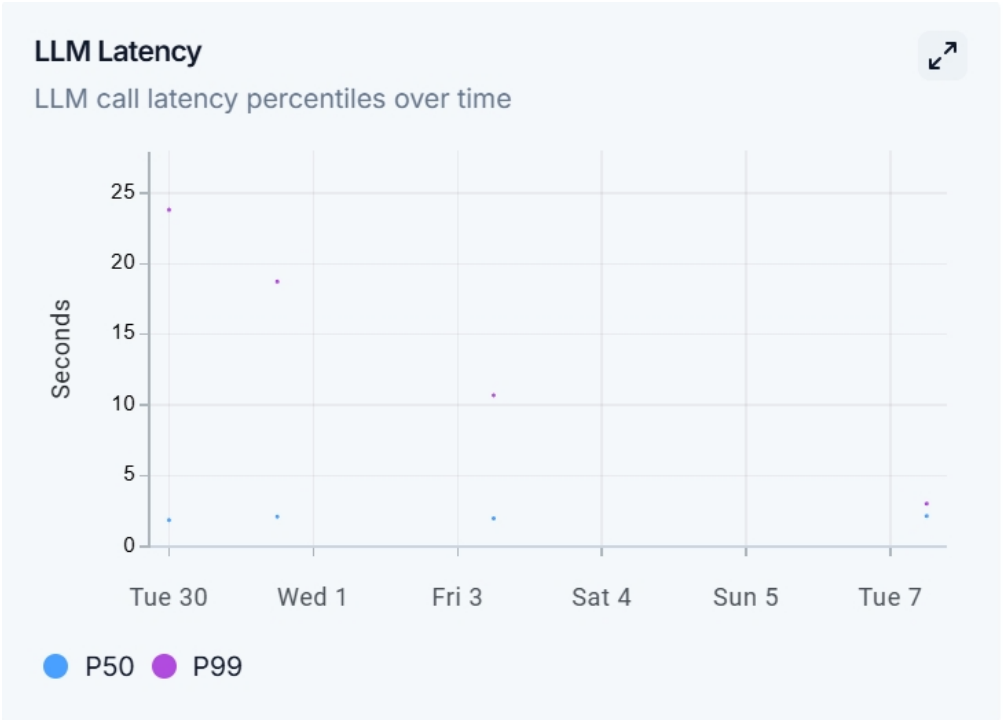


FIGURE 3.14 – Latence des appels LLM observée dans LangSmith : percentiles P50 et P99.

Le dernier export présente une latence médiane de 3,01 s et une latence moyenne de 4,53 s. Le p95 est de 10,56 s. Le maximum atteint 71,52 s sur un scénario data complexe, mais ce cas reste un outlier : la table retournée était correcte et aucune erreur SQL interne n'a été détectée. L'enjeu prioritaire n'est donc pas une instabilité générale, mais la surveillance de quelques cas coûteux côté worker ou API.

Latence par nœud du graphe : L'analyse par nœud confirme que les nœuds déterministes sont très peu coûteux. `input_guard` et `finalize_from_worker` ont une latence quasi négligeable par rapport au nœud `run_worker`. Ce résultat valide le choix d'architecture : les garde-fous déterministes et la finalisation structurée ajoutent peu de coût, tandis que la latence principale reste concentrée dans les appels workers, qui peuvent eux-mêmes appeler un LLM ou un tool.

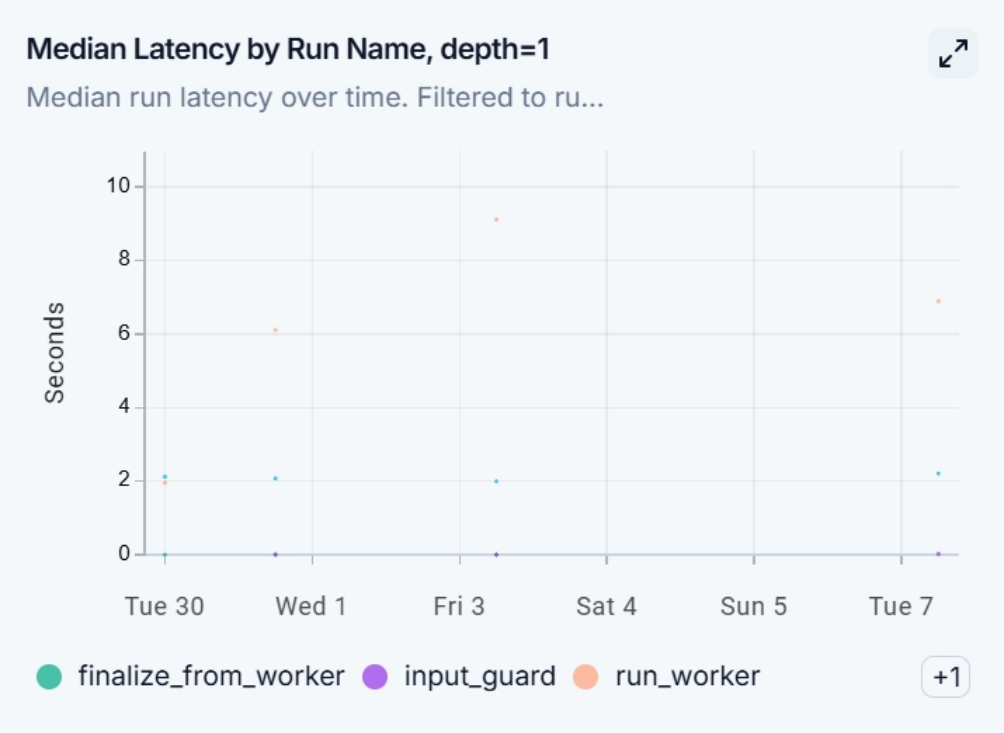


FIGURE 3.15 – Latence médiane par nœud LangGraph du supervisor.

Structure d'exécution du graphe : La courbe des appels de nœuds montre que `supervisor_decision` est le nœud central du graphe : il intervient à chaque tour pour décider de répondre directement ou d'appeler un worker. Les nœuds `run_worker`, `input_guard` et `finalize_from_worker` suivent la structure attendue du flux LangGraph. Cette observation confirme que LangSmith permet non seulement de mesurer des métriques globales, mais aussi d'inspecter la forme réelle de l'exécution.

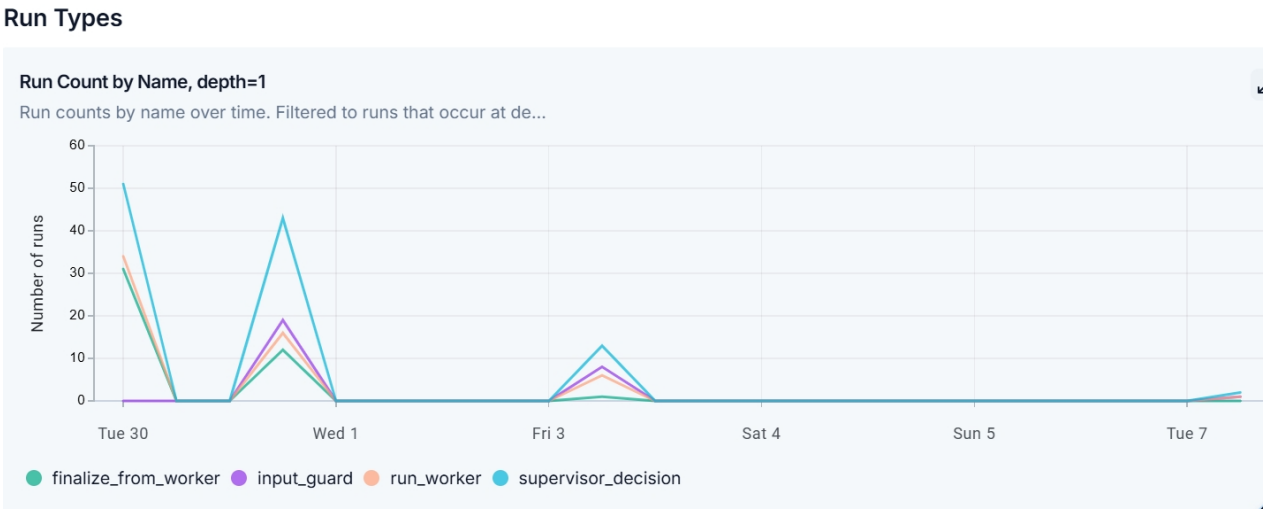


FIGURE 3.16 – Nombre d'appels par nœud LangGraph dans les traces LangSmith.

Tokens et coût : La consommation de tokens d'entrée diminue sur les derniers runs, avec un P99 plus faible que lors des essais précédents. Cette baisse correspond à la logique retenue dans le supervisor : limiter le contexte transmis aux workers, éviter de recopier les tableaux dans la réponse textuelle et ne pas refaire passer systématiquement les résultats workers par un second appel LLM.

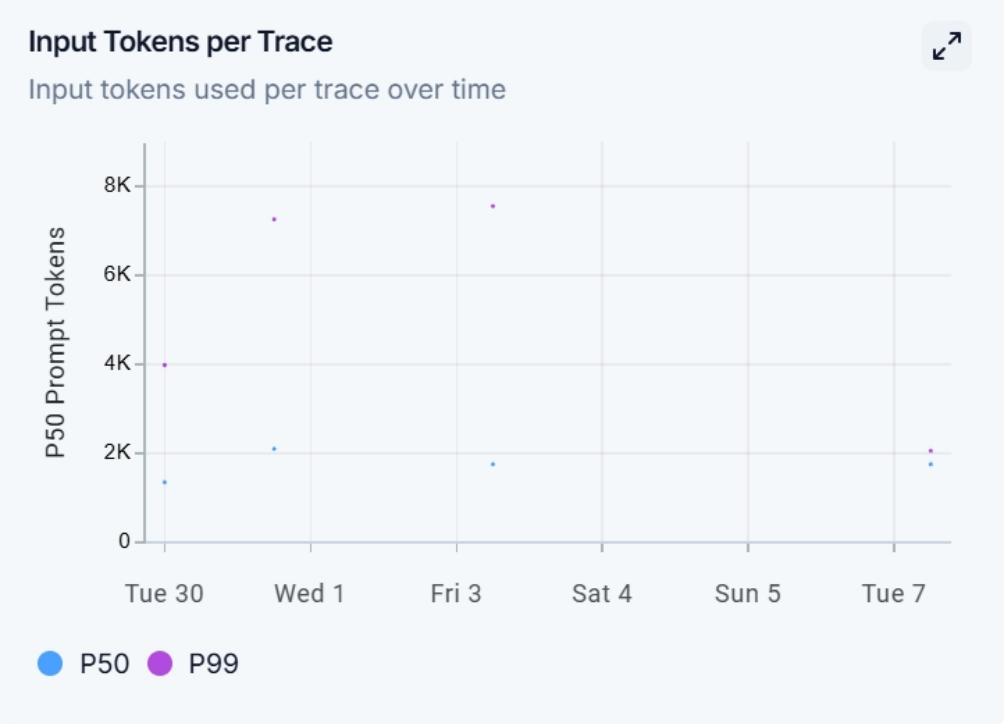


FIGURE 3.17 – Tokens d'entrée par trace LangSmith.

Le coût par trace reste très faible sur l'ensemble des runs. Le dernier export indique un coût moyen estimé de 0,000839 USD par exécution et un coût total estimé de 0,045331 USD pour 54 exécutions. Pour une passe de 27 demandes, le coût agent est estimé à environ 0,022666 USD.

Ce coût correspond aux appels modèles tracés dans LangSmith. Il n'inclut pas nécessairement l'ensemble des coûts d'infrastructure, de stockage, de monitoring, d'indexation documentaire ou d'exploitation cloud qui seraient à intégrer dans une analyse de coût complète.

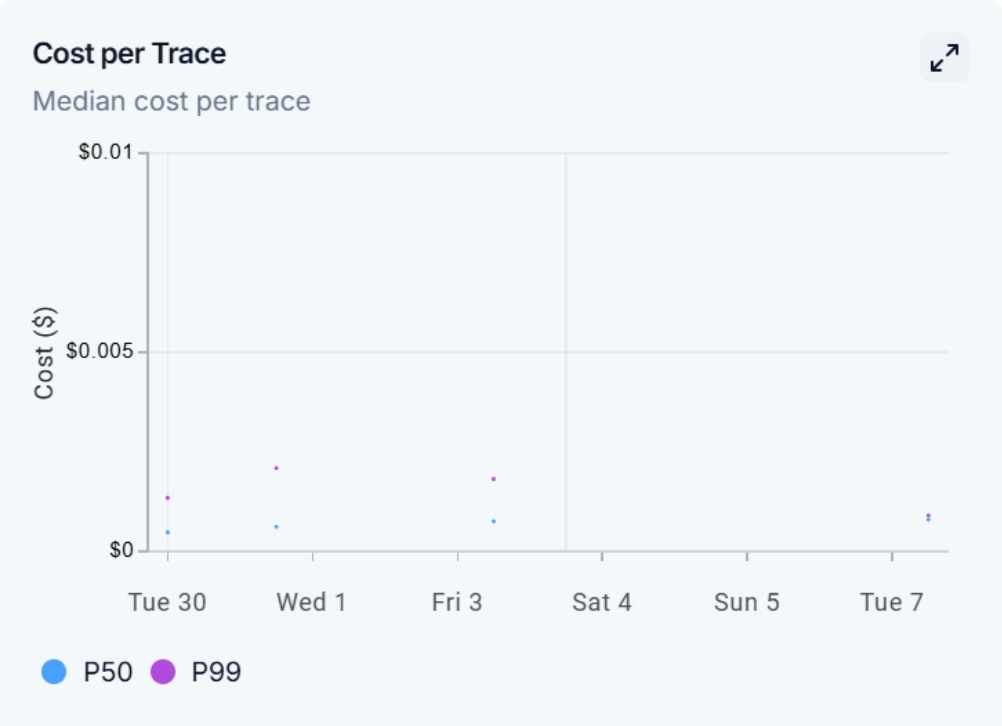


FIGURE 3.18 – Coût médian par trace dans LangSmith.

Résultats par domaine : Le worker `data` est le plus stable : 20/20 exécutions data passent fonctionnellement, et 22/22 sorties `agent_sql` contiennent une table exploitable lorsque l'on inclut le scénario de routage. Les workers `project` et `bdc` obtiennent respectivement 10/14 et 10/12 en score fonctionnel structurel. Les écarts restants ne remettent pas en cause l'architecture générale : ils concernent surtout la reprise de contexte multi-tours, par exemple lorsqu'un utilisateur ajoute une information courte à un brouillon projet ou BDC déjà en cours.

Lecture opérationnelle : Pour une passe complète de 27 demandes, le temps agent observé est d'environ 2,04 minutes. L'estimation manuelle, comprise entre 2 h 40 et 5 h 26 pour les mêmes demandes, est indicative et personnelle : elle repose sur mon estimation du temps nécessaire pour rechercher, vérifier et saisir les informations pour chaque scénario, à partir de ma compréhension du projet et des tâches réalisées pendant le stage. Elle ne constitue ni une mesure officielle validée par l'entreprise, ni une mesure expérimentale chronométrée auprès d'un panel d'utilisateurs. Elle n'inclut pas forcément tout le temps de vérification humaine qui resterait nécessaire avant une action engageante. Le résultat doit donc être lu comme un ordre de grandeur du gain potentiel, plus que comme une mesure définitive. Le gain principal n'est pas seulement le coût modèle tracé, mais la réduction des frictions : moins de navigation entre outils, moins d'allers-retours, accès plus rapide aux données et préparation plus guidée des formulaires projet ou BDC.

3.2.3 Discussion des contributions

La contribution principale du stage ne correspond pas à une seule fonctionnalité isolée, mais à une montée en maturité progressive du POC Spielberg. Le travail a commencé par une phase de cadrage et de compréhension des données, puis s'est déplacé vers le développement applicatif, avant de se concentrer sur la couche d'agents IA. Cette progression est importante car elle montre que les choix techniques n'ont pas été faits de manière abstraite : ils répondent à des problèmes rencontrés au fur et à mesure de l'implémentation.

Sur le plan data, la migration des fichiers JSON vers SQLite a permis de sortir d'une logique de mocks dispersés pour construire une base locale plus exploitable. Ce changement a rendu possibles des requêtes structurées, des jointures plus fiables, des endpoints FastAPI plus propres et un worker `data` capable d'interroger les données de manière contrôlée. Pour un POC, SQLite reste un compromis pertinent : léger, local, simple à versionner et suffisant pour valider le modèle relationnel avant une migration future vers PostgreSQL.

Sur le plan applicatif, le backend FastAPI, les schémas Pydantic, les services métier et les repositories ont permis de séparer progressivement les responsabilités. Les interfaces projet et BDC ne manipulent pas directement les données : elles passent par des contrats d'entrée validés côté backend. Cette séparation est importante dans un contexte où le frontend, les workflows métier et les agents IA peuvent produire des payloads structurés. Le même principe se retrouve dans l'intégration Kissflow : la création d'une demande BDC passe par un schéma, un mapper, un client API et une persistance locale, ce qui rend le flux plus traçable.

La partie la plus importante reste l'évolution de l'assistant conversationnel. Le passage du chat déterministe au premier agent LangChain a apporté une meilleure compréhension du langage naturel, mais a rapidement montré les limites d'un agent unique : prompt trop chargé, responsabilités mélangées, sorties frontend plus fragiles et difficulté à gérer des workflows multi-tours. L'architecture Router a constitué une première réponse, mais elle restait insuffisante lorsque l'utilisateur changeait de sujet ou complétait un brouillon déjà ouvert.

Le passage au Supervisor LangGraph représente donc la contribution technique la plus importante. Cette architecture conserve un seul interlocuteur côté utilisateur, tout en déléguant les tâches à des workers spécialisés. Elle permet de séparer le raisonnement LLM, les garde-fous, l'état conversationnel, les brouillons, les actions en attente et la finalisation déterministe. Les résultats LangSmith confirment l'intérêt de cette évolution : 54/54 exécutions en succès technique, 88,9 % de score fonctionnel structurel, 22/22 sorties SQL avec table et aucune erreur SQL interne détectée.

Les contributions peuvent être résumées comme une montée en maturité progressive : passage d'un cadrage data à une base locale exploitable, d'un backend de démonstration à une API plus structurée, d'interfaces simples à des écrans cohérents avec le design system, puis d'un chat déterministe à des agents. Cette progression s'est prolongée avec le passage d'un agent unique à une architecture Supervisor avec workers spécialisés, l'ajout d'un workflow externe réel avec Kissflow, l'ajout d'un worker RAG documentaire et la mise en place d'une démarche d'observabilité et d'évaluation avec LangSmith.

Les limites restantes sont également identifiées. Le système doit encore être consolidé sur la persistance longue durée des conversations, la reprise de contexte multi-tours, l'évaluation automatisée du RAG, le contrôle fin des droits d'accès et la migration vers une infrastructure cible plus industrielle. Ces limites ne remettent pas en cause l'intérêt du POC ; elles clarifient plutôt les étapes nécessaires pour passer d'une démonstration fonctionnelle à une solution plus robuste.

Chapitre 4

Conclusion et perspectives

4.1 Conclusion

Ce stage a permis de contribuer à un POC complet mêlant data engineering, backend, frontend, intégration API externe et intelligence artificielle appliquée. Le projet Spielberg illustre une idée centrale : un assistant IA en entreprise ne se résume pas à un prompt. Il doit être relié à des données structurées, à des API gouvernées, à des workflows contrôlés, à une interface réellement utilisable et à une politique de sécurité explicite.

Le travail réalisé a permis de faire évoluer Spielberg depuis une base de démonstration vers une application plus structurée. Les données ont été migrées vers SQLite, le backend a été organisé autour de FastAPI, Pydantic, services et repositories, les interfaces métier de création de projet et de demande BDC ont été développées, et l'intégration Kissflow a ajouté un workflow externe concret. En parallèle, la couche conversationnelle a connu une évolution progressive : chat déterministe, agent LangChain, Router, puis Supervisor LangGraph avec workers spécialisés.

La contribution la plus importante concerne l'architecture agentique. Le Supervisor répond aux limites observées dans les étapes précédentes : il garde un seul fil conversationnel, isole les workers, contrôle les actions sensibles et préserve les payloads frontend. L'évaluation LangSmith montre que cette approche est fonctionnelle sur le périmètre testé, avec une stabilité technique complète et un score fonctionnel structurel de 88,9 %. Le POC reste perfectible, mais il démontre déjà une valeur opérationnelle : réduction du temps nécessaire pour consulter des données, préparer des formulaires et accéder à la documentation interne.

Au-delà du résultat technique, ce stage a également montré l'importance du cadrage, de la documentation, de la conformité et de la communication avec les équipes métier. Les réunions et ateliers réguliers, les échanges avec différents pôles, les formations Azure et conformité, ainsi que la veille sur les LLM, le RAG, LangChain et LangGraph ont permis de relier les décisions d'architecture à des besoins concrets.

4.2 Enseignements tirés

Les principaux apprentissages du stage concernent l'importance du cadrage métier avant le code, le rôle central des contrats de données, les limites d'un chat déterministe et la valeur mais aussi les risques des agents LLM. Le stage a également montré la nécessité de justifier les patterns d'architecture, l'importance du design system pour rendre un POC présentable, l'utilité de mesurer la latence et la qualité des réponses, ainsi que le besoin de garder la sécurité et la conformité dès le début du projet.

Sur le plan technique, le stage m'a appris qu'un système IA utile en entreprise est d'abord un système logiciel bien conçu. La qualité du prompt ne suffit pas : il faut des schémas, des garde-fous, des tests, une observabilité, une séparation claire entre décision probabiliste et exécution déterministe, ainsi qu'une interface capable d'exploiter les sorties structurées.

Sur le plan humain, le stage a montré l'importance de comprendre les contraintes des utilisateurs

avant de chercher une solution technique. Les échanges avec les équipes métier, les ateliers réguliers et les retours progressifs sur le POC ont permis d'ajuster les priorités : rendre l'outil compréhensible, éviter les actions automatiques risquées, garder une trace des décisions et construire une architecture que l'équipe pourra continuer à faire évoluer.

4.3 Prochaines étapes

Les prochaines étapes possibles peuvent être regroupées autour de quelques axes :

— compléter les tests automatisés sur le backend, les workers et les workflows multi-tours ;
— enrichir l'évaluation LangSmith avec davantage de scénarios RAG et de reprises de contexte ;
— migrer progressivement de SQLite vers PostgreSQL, remplacer l'`InMemorySaver` par un checkpointer durable et persister les conversations par utilisateur ;
— renforcer le RAG avec du reranking, de la recherche hybride, une meilleure extraction PDF et une gestion plus fine des droits documentaires ;
— préparer l'industrialisation Azure dans le cadre de l'architecture validée par GSO et RE :SOURCES : séparation Front / App / Data, réseaux privés, secrets, monitoring, RBAC, audit, data lineage et suivi de l'impact environnemental des appels LLM.

À plus long terme, l'exposition éventuelle de certains outils via MCP et la connexion progressive aux vraies sources Groupe permettraient de rapprocher Spielberg d'une plateforme réellement industrialisable.

Annexe A

DD&RS - Développement durable et responsabilité sociale

A.1 Impacts environnementaux

A.1.1 Liés à l'activité de l'entreprise

Prodigious France appartient à l'écosystème Publicis Groupe. L'activité du Groupe se situe dans les services de communication, de production, de data, de technologie et de marketing. Les impacts environnementaux directs et indirects sont donc principalement liés aux bureaux, aux déplacements, aux outils numériques, à la production de contenus, aux plateformes cloud, au stockage de fichiers et aux campagnes réalisées pour les clients.

Publicis Groupe structure sa politique RSE autour de trois priorités : l'impact et l'équité sociale, le marketing responsable et l'éthique des affaires, ainsi que la lutte contre le changement climatique [2, 3]. Sur le volet environnemental, la stratégie officielle mentionne une ambition climatique validée par SBTi, avec un objectif de réduction de 50 % des émissions carbone à horizon 2030 et une trajectoire Net Zero à horizon 2040. La page RSE du Groupe indique également près de 80 % d'énergie renouvelable issue de sources directes en 2025 [2].

A.1.2 Liés au fonctionnement de l'entreprise

À l'échelle de l'entreprise, les impacts proviennent notamment de la consommation énergétique des bureaux, du matériel informatique, des environnements collaboratifs, des outils SaaS, du stockage documentaire et des infrastructures utilisées pour la production et l'analyse de données. Publicis Groupe mentionne plusieurs dispositifs liés à la responsabilité environnementale et au marketing responsable, dont A.L.I.C.E, un calculateur carbone personnalisé disponible dans de nombreux pays, et N.I.B.I. (*No Impact for Big Impact*), un programme lié à la durabilité et au marketing responsable [2, 3].

A.1.3 Liés à l'activité de l'équipe ou du département

Dans le cadre du projet Spielberg, l'impact principal vient de l'usage numérique : stockage de données, appels LLM, index vectoriel, logs, environnements de développement, tests et éventuelle future infrastructure cloud. Les appels aux modèles de langage ont un coût économique et environnemental indirect, car ils consomment des ressources de calcul. Le projet doit donc limiter les appels inutiles, réduire les boucles LLM, éviter la duplication de contexte et réutiliser les composants lorsque c'est possible.

Cet impact reste limité à l'échelle d'un POC, mais il est important de l'intégrer dès la conception, car le coût peut augmenter rapidement si l'assistant est utilisé par plusieurs équipes, avec davantage de documents, de traces LangSmith et d'appels API.

A.2 Actions pour minimiser les impacts

A.2.1 Actions mises en place par l'entreprise

Les actions RSE identifiées côté Publicis Groupe s'inscrivent dans les trois priorités officielles : climat, impact social et éthique des affaires. La stratégie RSE publiée par le Groupe mentionne notamment une ambition climatique validée par SBTi, un objectif de réduction de 50 % des émissions carbone à horizon 2030, une trajectoire Net Zero à horizon 2040, l'augmentation de la part d'énergie renouvelable, l'intégration des enjeux climatiques dans les pratiques professionnelles avec A.L.I.C.E, le programme N.I.B.I. pour la durabilité et le marketing responsable, ainsi que des politiques et chartes internes relatives à l'éthique, la lutte anti-corruption, la protection des données, la sécurité de l'information, les droits humains et le signalement [4].

A.2.2 Actions intégrées dans les stratégies de l'entreprise

Les formations internes suivies pendant le stage, notamment protection des données, conformité et anti-bribery, participent à cette stratégie globale de responsabilité. Elles rappellent que la performance d'un projet data ou IA ne peut pas être séparée des enjeux de gouvernance, confidentialité, sécurité, lutte contre la corruption et usage responsable des informations.

Dans le contexte Spielberg, ces principes se traduisent par une attention particulière portée à la minimisation des données exposées au LLM, à la non-divulgation des secrets, à la validation humaine des actions sensibles et à la traçabilité des traitements.

A.2.3 Canaux de communication dans l'environnement de travail

La communication autour du projet s'est faite via réunions et ateliers réguliers, échanges avec différents pôles, documentation technique, supports de cadrage et tickets de suivi. Certains échanges impliquaient aussi des interlocuteurs internationaux. Cette organisation a permis d'ajuster progressivement le périmètre, de clarifier les besoins métier et de valider les choix fonctionnels.

A.2.4 Actions liées au stage et aux stratégies de l'entreprise

Dans le POC, plusieurs choix vont dans le sens d'une approche responsable : limitation des données envoyées au LLM, safe mode par défaut, input guard, réduction des appels LLM inutiles, architecture évitant certaines boucles coûteuses et documentation destinée à faciliter la reprise du projet sans gaspillage de travail.

Plusieurs décisions techniques du projet contribuent directement à cette logique. Les singletons et caches évitent de recréer inutilement certains objets coûteux : agent supervisor, workers, vector store Chroma, chargement des données et cache temporaire sur les listes de projets. Le nœud `finalize_from_worker` évite un second appel LLM lorsque le résultat du worker est déjà exploitable par le frontend. Le worker RAG limite le nombre de sources récupérées à quatre passages, ce qui évite d'envoyer un contexte documentaire trop volumineux au modèle. Enfin, les règles du frontend évitent de recopier dans la réponse textuelle les tableaux déjà affichés sous la réponse, ce qui réduit aussi les tokens.

Ces choix restent techniques, mais ils sont cohérents avec une démarche DD&RS appliquée au numérique : moins d'appels inutiles, moins de contexte transmis, plus de traçabilité, plus de sécurité et une meilleure maintenabilité.

Annexe B

Structure de l'entreprise

Prodigious France appartient à Publicis Groupe. Le projet Spielberg se situe à l'intersection de plusieurs fonctions : production, opérations, finance, ressources humaines, IT, sécurité, data et IA.

Publicis Groupe est un groupe international de communication, marketing, data et technologie. Ci-dessous se trouve l'organigramme du groupe Publicis. Pour rappel, j'occupais le poste d'Assistant Lead Automation avec les initiales M.C en bas à gauche de la figure.

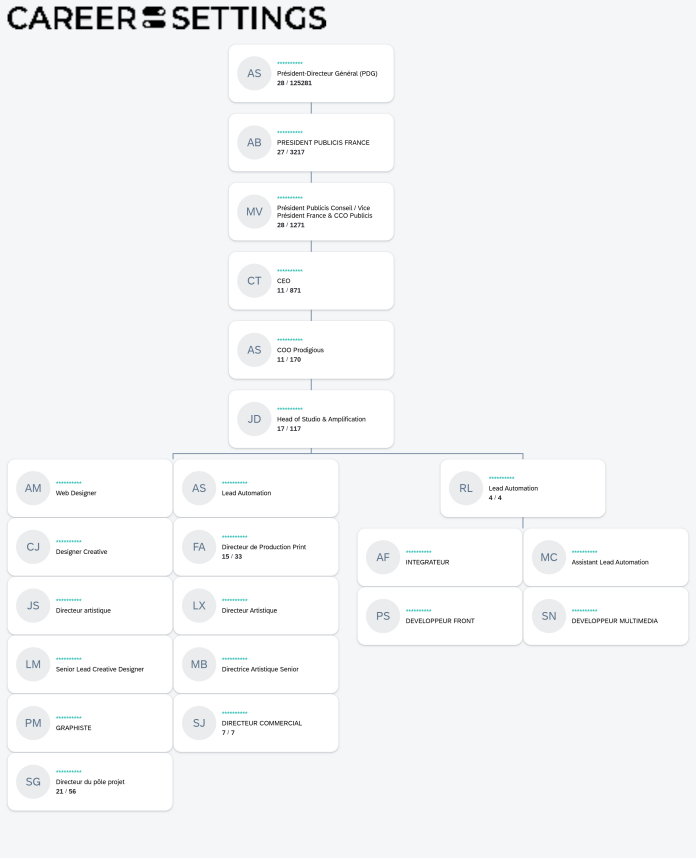


FIGURE B.1 – Organigramme Prodigious/Publicis.

Annexe C

Détails techniques

Cette annexe regroupe des extraits de code représentatifs des choix techniques du projet. Ils ne visent pas à reproduire tout le code, mais à documenter les mécanismes structurants : cache, singletons, garde-fous, accès SQL contrôlé, finalisation déterministe, RAG et intégration externe.

C.1 Cache et réduction des chargements inutiles

Le chargement des données Teamium et des mocks est mis en cache avec `lru_cache`. Cette décision évite de relire les fichiers Excel et JSON à chaque requête, ce qui réduit la latence et la consommation de ressources en développement local.

```python
@lru_cache(maxsize=1)
def load_teamium_bookings(
    path: str = str(DATA_XLSX_DEFAULT),
    sheet: str = DEFAULT_SHEET,
) -> pd.DataFrame:
    df = pd.read_excel(path, sheet_name=sheet, header=0)
    return _clean_teamium_df(df)

@lru_cache(maxsize=1)
def load_mock_datasets(path: str = str(MOCKS_DIR)) -> Dict[str, Any]:
    ...
```

Listing C.1 – Cache des chargements de données dans `data_loader.py`.

C.2 Cache applicatif sur les projets

La consultation de la liste des projets utilise un cache court côté service. Le cache est invalidé lors de la création ou de la modification d'un projet. Ce mécanisme améliore la réactivité du frontend sans rendre les données durablement obsolètes.

```python
PROJECTS_LIST_CACHE_TTL_SECONDS = 60
_PROJECTS_LIST_CACHE: dict[tuple[str, str, str, str, int], tuple[float, dict[str, Any
    ]]] = {}

def clear_projects_list_cache() -> None:
    _PROJECTS_LIST_CACHE.clear()

def list_projects_from_db(...):
    cache_key = _projects_list_cache_key(status, client, department, q, limit)
    cached = _PROJECTS_LIST_CACHE.get(cache_key)
    if cached and time.monotonic() - cached[0] < PROJECTS_LIST_CACHE_TTL_SECONDS:
        return cached[1]

    ...
    _PROJECTS_LIST_CACHE[cache_key] = (time.monotonic(), result)
```

```
    return result
```

Listing C.2 – Cache court pour la liste des projets dans `services_projects.py`.

C.3 Singleton du supervisor

Les agents LLM sont créés paresseusement puis réutilisés. Le singleton du supervisor évite de reconstruire le client modèle, le prompt et la configuration à chaque message utilisateur.

```
_agent = None

def get_supervisor_agent():
    global _agent

    if _agent is None:
        _agent = create_agent(
            model=ChatOpenAI(model=os.getenv("OPENAI_MODEL", "gpt-4.1-mini")),
            system_prompt=SUPERVISOR_PROMPT,
            response_format=ToolStrategy(SupervisorDecision),
            checkpointer=InMemorySaver(),
        )

    return _agent
```

Listing C.3 – Création paresseuse du supervisor dans `supervisor/agent.py`.

C.4 Garde-fous avant appel LLM

L'`input_guard` applique des règles déterministes avant l'appel au supervisor. Il bloque les injections de prompt, les demandes de secrets, l'accès direct à la base, le SQL brut, les exports massifs et certaines données RH sensibles.

```
GUARD_RULES: tuple[GuardRule, ...] = (
    GuardRule(
        blocked_reason="prompt_injection",
        severity="critical",
        patterns=(
            r"\bignore\b.*\b(instructions?|rules?|system|developer|previous|prior)\b",
            r"\b(disregard|bypass|override)\b.*\b(instructions?|rules?|policy|system)\b",
            r"\b(jailbreak|developer mode|mode developpeur)\b",
        ),
    ),
    GuardRule(
        blocked_reason="reveal_secrets",
        severity="critical",
        patterns=(
            r"\b(secret|token|password|credential)\b",
            r"\b(mot de passe|cle secrete|identifiants? secrets?)\b",
        ),
    ),
)
```

Listing C.4 – Extrait des règles de garde-fous dans `supervisor/input_guard.py`.

C.5 Accès SQL read-only

Le worker data ne dispose pas d'un accès SQL libre. Le tool SQL vérifie que la requête est en lecture seule et bloque les instructions destructives ou de modification.

```
READ_ONLY_STARTS = ("select", "pragma", "with")
FORBIDDEN_SQL = re.compile(
    r"\b(insert|update|delete|drop|alter|create|truncate|replace|attach|detach|vacuum
    |reindex)\b",
    re.IGNORECASE,
)

def is_read_only_sql(query: str) -> bool:
    cleaned = query.strip().rstrip(";").lower()
    if not cleaned:
        return False
    if FORBIDDEN_SQL.search(cleaned):
        return False
    return cleaned.startswith(READ_ONLY_STARTS)
```

Listing C.5 – Restriction SQL read-only dans `agent_sql.py`.

C.6 Finalisation déterministe après worker

Le nœud `finalize_from_worker` transforme le résultat d'un worker en réponse frontend sans repasser systématiquement par un second appel LLM. Ce choix réduit la latence, les tokens et le risque de perdre les tableaux ou formulaires structurés.

```
def finalize_from_worker_node(state: SupervisorState) -> dict[str, Any]:
    worker_results = state.get("worker_results") or []
    latest_worker_result = worker_results[-1] if worker_results else {}

    worker_name = latest_worker_result.get("worker", "worker")
    worker_data = latest_worker_result.get("data") or {}

    response = SpielbergResponse(
        mode="supervisor",
        answer=latest_worker_result.get("summary", "(pas de reponse finale)"),
        tool_used=worker_name,
        tools_used=["supervisor", worker_name],
        evidence=latest_worker_result.get("evidence", []),
        data=AssistantData(**worker_data),
    )
    return {"final_response": response_dict(response)}
```

Listing C.6 – Finalisation déterministe dans `supervisor/graph.py`.

C.7 RAG et vector store persistant

Le module RAG utilise Chroma comme base vectorielle locale persistante. Le vector store est créé paresseusement et réutilisé, ce qui évite de reconstruire l'objet à chaque recherche documentaire.

```
_vector_store = None

def get_vector_store() -> Any:
    global _vector_store
```

```
    if _vector_store is None:
        RAG_INDEX_DIR.mkdir(parents=True, exist_ok=True)
        _vector_store = Chroma(
            collection_name=RAG_COLLECTION_NAME,
            embedding_function=get_embeddings(),
            persist_directory=str(RAG_INDEX_DIR),
        )

    return _vector_store

def search_document(question: str, *, k: int = 4) -> list[Any]:
    vector_store = get_vector_store()
    return vector_store.similarity_search(question, k=k)
```

Listing C.7 – Vector store Chroma réutilisé dans `rag/rag.py`.

C.8 Worker RAG et sources frontend

Le worker RAG retourne toujours des données structurées : cards, tableau de sources, données brutes et evidence. Le frontend peut donc afficher les sources sous la réponse, sans que le supervisor les recopie dans le texte.

```
retrieved_rows = [
    {
        "source": doc.metadata.get("source_name") or doc.metadata.get("source"),
        "page": doc.metadata.get("page"),
        "chunk_id": doc.metadata.get("chunk_id"),
        "preview": doc.page_content[:500],
    }
    for doc in docs
]

return {
    "worker": "rag",
    "status": "success",
    "summary": summary,
    "data": {
        "kind": "rag",
        "cards": [
            {"label": "Sources", "value": str(len(docs)), "meta": "RAG"},
            {"label": "Indexed", "value": str(indexed_count), "meta": "chunks"},
        ],
        "table": {"title": "RAG Sources", "rows": retrieved_rows},
        "raw": {"retrieved": retrieved_rows},
    },
    "evidence": retrieved_rows,
}
```

Listing C.8 – Sortie structurée du worker RAG dans `supervisor/rag_worker.py`.

C.9 Intégration BDC et traçabilité

La création BDC génère un identifiant Spielberg, mappe le payload vers Kissflow, soumet la demande et conserve une trace locale. Ce flux sépare compréhension, validation, mapping, appel externe et persistance.

```python
async def create_submit_bdc_request(bdc: BDCFrontCreateRequest) -> dict:
    internal_bdc = build_bdc_create_request(bdc)
    payload = map_bdc_create_to_kissflow(internal_bdc)

    created = await get_kissflow_client().create_item(payload)
    instance_id = created.get("_id")
    activity_instance_id = created.get("_activity_instance_id")

    result = await get_kissflow_client().submit_item(instance_id,
    activity_instance_id)
    insert_bdc_request(internal_bdc, created)

    return {
        "Spielberg_Request_ID": internal_bdc.Spielberg_Request_ID,
        "request_status": internal_bdc.request_status.value,
        "kissflow": result,
    }
```

Listing C.9 – Création et soumission BDC dans `services_bdc.py`.

Bibliographie

[1] Prodigious France. *CDC_Spielberg_v1.0 : Projet Spielberg, plateforme de données et assistant IA interne*. Document interne, mai 2026.

[2] Publicis Groupe. *Corporate Social Responsibility*. Page officielle consultée en juillet 2026. https://www.publicisgroupe.com/en/the-groupe/corporate-social-responsibility/csr-strategy

[3] Publicis Groupe. *CSR strategy - Creation of sustainable value*. Transcription officielle de l'infographie RSE, consultée en juillet 2026. https://www.publicisgroupe.com/en/the-groupe/corporate-social-responsibility/csr-strategic-creation-of-sustainable-value

[4] Publicis Groupe. *CSR Smart Data Library*. Bibliothèque officielle des documents RSE, éthique, conformité, data privacy et politiques internes, consultée en juillet 2026. https://publicisgroupe-csr-smart-data.com/en/links

[5] Anthropic. *Building effective agents*. 2024.

[6] Anthropic. *Introducing the Model Context Protocol*. 2024.

[7] LangChain. *Multi-agent systems*. Documentation officielle.

[8] LangChain. *Subagents*. Documentation officielle. https://docs.langchain.com/oss/python/langchain/multi-agent/subagents

[9] LangChain. *Choosing the Right Multi-Agent Architecture*. LangChain Blog, 2026. https://www.langchain.com/blog/choosing-the-right-multi-agent-architecture

[10] LangChain. *Benchmarking Multi-Agent Architectures*. LangChain Blog, 2026. https://www.langchain.com/blog/benchmarking-multi-agent-architectures

[11] LangChain. *Agents and tool calling*. Documentation officielle.

[12] LangGraph. *Building stateful multi-actor applications with LLMs*. Documentation officielle.

[13] Hugging Face. *Hugging Face Course : Natural Language Processing, Transformers and LLMs*. Cours en ligne.

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, ... : *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.

[15] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, Wen-tau Yih. *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP, 2020.

[16] Nils Reimers, Iryna Gurevych. *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*. EMNLP, 2019.

[17] Shunyu Yao et al. *ReAct : Synergizing Reasoning and Acting in Language Models*. arXiv :2210.03629, 2022.

[18] Timo Schick et al. *Toolformer : Language Models Can Teach Themselves to Use Tools*. arXiv :2302.04761, 2023.

[19] Chroma. *Chroma Documentation*. Documentation officielle.

[20] Mouhamed Cisse. *Rapport de refactoring agentique : de l'orchestrator au supervisor Spielberg*. Document interne du projet, 2026.

[21] Kissflow. *Kissflow API Documentation*. Documentation officielle Kissflow, consultée pendant l'intégration du workflow BDC.